

Algorytmy sortowania - analiza porównawcza

Piotr Niepsuj

Spis treści

1. Wprowadzenie	2
1.1. Środowisko testowe i sprzęt	2
1.2. Metoda generowania danych testowych	2
1.3. Metoda testowania	3
2. Algorytmy prostsze	4
2.1. Insertion sort	4
2.2. Selection sort	6
2.3. Bubble sort	8
2.4. Porównanie algorytmów prostszych	10
2.4.1. Dane losowe	10
2.4.2. Dane posortowane malejąco (odwrócone)	11
2.4.3. Dane posortowane rosnąco	12
2.4.4. Dane prawie posortowane (sąsiednia wymiana)	13
2.4.5. Dane prawie posortowane (globalna wymiana)	14
3. Algorytmy efektywniejsze	15
3.1. Quicksort	15
3.2. Shellsort	17
3.3. Heapsort	20
3.4. Porównanie algorytmów efektywniejszych	23
3.4.1. Dane losowe	23
3.4.2. Dane posortowane malejąco (odwrócone)	24
3.4.3. Dane posortowane rosnąco	25
3.4.4. Dane prawie posortowane (sąsiednia wymiana)	26
3.4.5. Dane prawie posortowane (globalna wymiana)	27
3.5. Porównanie algorytmów prostszych i efektywniejszych	28
3.5.1. Dane losowe	28
3.5.2. Dane posortowane malejąco (odwrócone)	29
3.5.3. Dane posortowane rosnąco	30
3.5.4. Dane prawie posortowane (sąsiednia wymiana)	31
3.5.5. Dane prawie posortowane (globalna wymiana)	32
4. Algorytmy niekonwencjonalne	33
4.1. Stoooge sort	33
4.2. Thanos sort	35
4.3. Stalin sort	37

1. Wprowadzenie

Głównym celem tego projektu jest praktyczne zbadanie, jak wybór algorytmu sortowania wpływa na czas wykonania programu. Zamiast opierać się tylko na samej teorii ze złożoności obliczeniowej, projekt zakłada sformułowanie własnych hipotez i ich empiryczną weryfikację. Kluczowym kryterium oceny wydajności poszczególnych metod będzie zmierzony czas ich działania.

W ramach projektu zaimplementowałem algorytmy podzielone na trzy kategorie:

- **Część I (algorytmy proste):** Insertion sort, Selection sort oraz Bubble sort.
- **Część II (algorytmy efektywniejsze):** Quicksort, Shellsort oraz Heapsort.
- **Część III (algorytmy niekonwencjonalne):** Stooge sort, Thanos sort oraz Stalin sort.

Aby uzyskać pełny obraz tego, jak algorytmy zachowują się w różnych warunkach, każdy z nich jest testowany na pięciu scenariuszach przygotowania danych:

1. **Dane losowe** - nasz główny punkt odniesienia pokazujący wydajność w „normalnych” warunkach.
2. **Dane posortowane malejąco (odwrócone)** - czyli układ odwrócony, często będący najgorszym przypadkiem dla wielu metod.
3. **Dane posortowane rosnąco** - optymistyczny scenariusz dla większości algorytmów.
4. **Dane prawie posortowane (sąsiednia wymiana)** - zbiór z około 10% zamian sąsiednich elementów, co imituje dane z małym bałaganem.
5. **Dane prawie posortowane (globalna wymiana)** - zbiór z około 10% zamian losowych elementów, co odpowiada sytuacji, gdy dane mają szum.

1.1. Środowisko testowe i sprzęt

- **Procesor:** Intel Core i5-12600K
- **Pamięć RAM:** 32 GB (DDR4)
- **Język programowania:** C++
- **Kompilator:** g++ (GCC) 14.3.0

1.2. Metoda generowania danych testowych

Do przygotowania danych wejściowych stworzyłem program w języku C++. Program generuje zestawy danych dla wielkości bazujących na potęgach liczby 10. Aby zwiększyć rozdzielczość wykresów, potęgi te są dodatkowo zagęszczane przez mnożniki 1, 2 oraz 5 (co daje nam tablice o rozmiarach np. 10, 20, 50, 100, 200, 500, 1000 itd.). Wygenerowane dane zapisywane są do plików CSV, dzięki czemu każdy algorytm operuje na dokładnie takich samych liczbach w danym scenariuszu.

Program obsługuje się z wiersza poleceń, a zakres generowanych danych można łatwo dostosować (przedziały podawane są łącznie). Przykłady użycia:

- **./main generate** - generuje domyślnie wszystkie dane wielkości od 1 do 8 potęgi 10.
- **./main generate 3 6** - ogranicza generowanie plików CSV tylko do rozmiarów od 3 do 6 potęgi 10.

1.3. Metoda testowania

Ten sam program w C++ odpowiada za przeprowadzanie właściwych pomiarów. Aplikacja wczytuje przygotowane wcześniej dane z plików CSV, uruchamia wybrany algorytm i mierzy jego czas wykonania (z wykorzystaniem wbudowanych narzędzi biblioteki chrono).

Interfejs z poziomu konsoli pozwala na bardzo dużą elastyczność w dobieraniu testów, co jest szczególnie przydatne przy wolniejszych algorytmach, których nie chcemy puszczać dla ogromnych tablic.

Oto jak w praktyce wygląda sterowanie testami:

- **./main test** - odpala komplet pomiarów: wszystkie algorytmy na wszystkich 5 scenariuszach w pełnym przedziale (potęgi 1-8).
- **./main test 1 4** - testuje wszystkie algorytmy i scenariusze, ale zawęża zestaw danych do potęg od 1 do 4.
- **./main test 1 7 quick-sort descending** - testuje wyłącznie algorytm Quicksort na danych posortowanych malejąco, dla potęg 1-7.
- **./main test 1 4 bubble-sort all** - uruchamia Bubble sort na wszystkich 5 scenariuszach dla potęg 1-4.
- **./main test 1 4 all descending** - sprawdza zachowanie wszystkich zaimplementowanych algorytmów, ale tylko na danych malejących, w rozmiarach od potęgi 1 do 4.

Pojedynczy test na danym algorytmie, scenariuszu i rozmiarze danych generuje plik CSV zawierający próby od -10 do 9 i czas wykonania algorytmu w nanosekundach, przykład można zobaczyć na Tabeli 1. Przy wyliczaniu średniej do wykresów tylko próby od 0 do 9 są brane pod uwagę. Próby od -10 do -1 są na rozgrzewkę dla procesora.

Każdy test zostaje przeprowadzony na odizolowanym rdzeniu procesora z regulatorem skalowania ustawionym na wydajność. W moim przypadku korzystam z rdzenia 0, który jest rdzeniem wydajnościowym. Testy są także włączane z najwyższym możliwym priorytetem. Zostało to osiągnięte w następujący sposób

- **Izolacja rdzenia 0**
Dodanie do /etc/default/grub ustawienia
GRUB_CMDLINE_LINUX_DEFAULT="isolcpus=0
nohz_full=0 rcu_nocbs=0"
- **Ustawienie regulatora skalowania dla rdzenia 0 na wydajność (do restartu systemu)**
Wykonane za pomocą komendy `echo performance | sudo tee /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor` można sprawdzić czy operacja powiodła się wypisując `cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor`
- **Uruchomienie testów na rdzeniu 0 z największym priorytetem** Wykorzystanie `taskset` do przypisania procesu do rdzenia 0 oraz `chrt` do ustawienia najwyższego priorytetu. Komenda użyta w testach to `sudo chrt -f 99 taskset -c 0 ./main test`

Próba	Czas wykonania (ns)
-10	11464
-9	7212
-8	5258
-7	4029
-6	3633
-5	3478
-4	3333
-3	3211
-2	3163
-1	3192
0	3154
1	3211
2	3164
3	3144
4	3143
5	3180
6	3161
7	3145
8	3161
9	3143

Tabela 1: Dane testowe pliku CSV dla Bubble sort na tablicy 100 elementowej z losowym ułożeniem elementów.

2. Algorytmy prostsze

2.1. Insertion sort

Zasada działania algorytmu

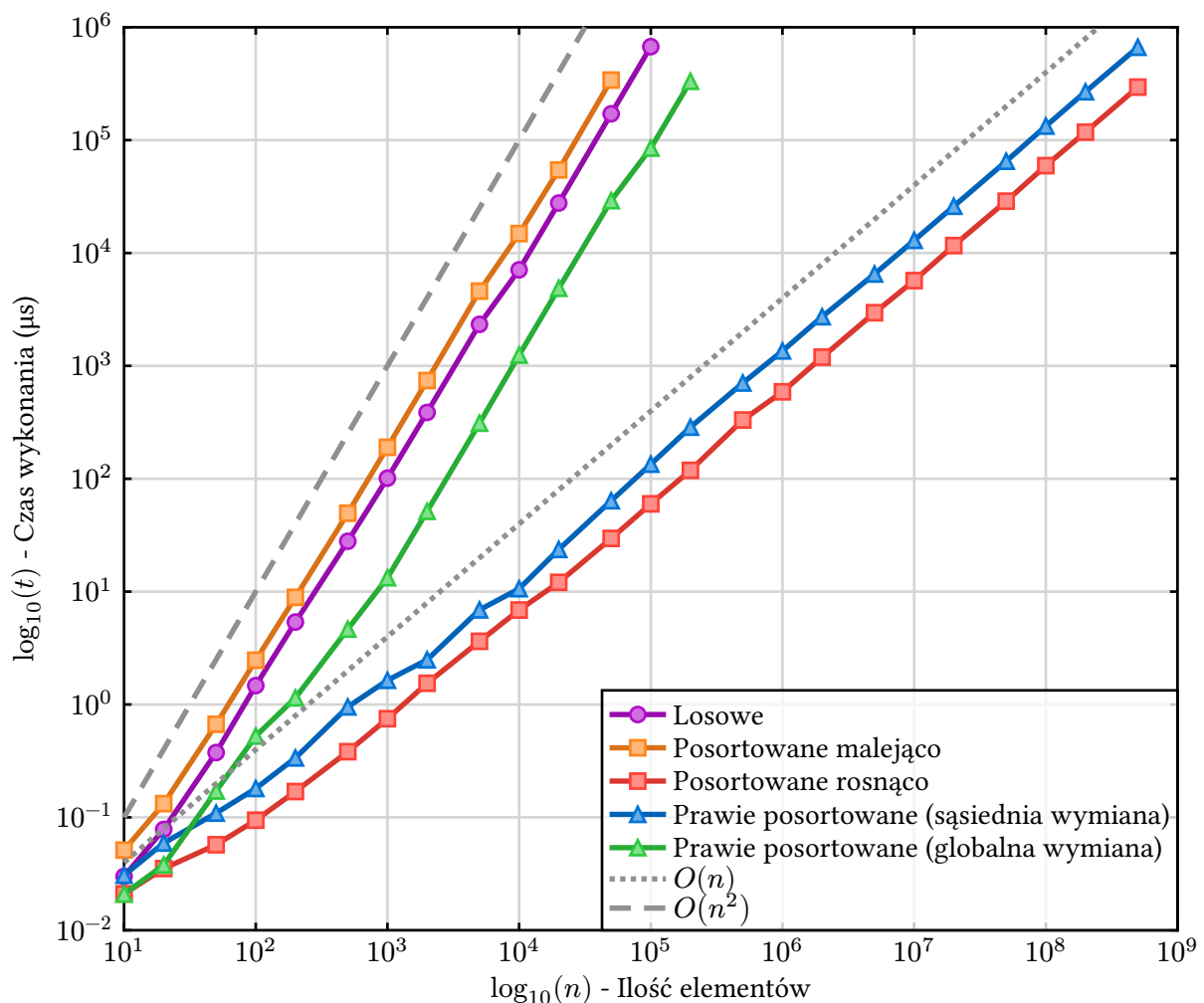
Sortowanie przez wstawianie działa bardzo intuicyjnie, dokładnie tak, jak większość ludzi układająca karty w ręce podczas gry. Algorytm dzieli tablicę na dwie części, jedną posortowaną (na początku) i drugą nieposortowaną (reszta). W każdej iteracji bierze pierwszy element z części nieposortowanej i szuka dla niego odpowiedniego miejsca w części posortowanej, przesuwając większe elementy w prawo, aby zrobić mu miejsce. Kiedy znajdzie właściwą pozycję, wstawia tam element. Proces ten powtarza się, aż cała tablica będzie uporządkowana.

Algorytm 1: Insertion sort

```
1: procedure INSERTION SORT(arr, n)
2:    $i \leftarrow 1$ 
3:   while  $i < n$  do
4:     current  $\leftarrow$  arr[ $i$ ]
5:      $j \leftarrow i - 1$ 
6:     while  $j \geq 0$  and arr[ $j$ ] > current do
7:       arr[ $j + 1$ ]  $\leftarrow$  arr[ $j$ ]
8:        $j \leftarrow j - 1$ 
9:     end
10:    arr[ $j + 1$ ]  $\leftarrow$  current
11:     $i \leftarrow i + 1$ 
12:  end
13: end
```

Hipotezy badawcze dla poszczególnych scenariuszy

1. **Dane losowe:** Oczekuję czasu rzędu $O(n^2)$. Z powodu dużej liczby porównań i przesunięć, algorytm będzie działał stosunkowo wolno dla większych rozmiarów tablic.
2. **Dane posortowane malejąco (odwrócone):** worst-case - Każdy nowy element będzie musiał zostać przesunięty na sam początek tablicy. Czas wykonania powinien być najdłuższy ze wszystkich scenariuszy.
3. **Dane posortowane rosnąco:** best-case - Złożoność spada do $O(n)$, ponieważ warunek wewnętrznej pętli nigdy nie zostanie spełniony. Oczekuję wykresu liniowego.
4. **Dane prawie posortowane (sąsiednia wymiana):** Ponieważ Insertion sort świetnie radzi sobie, gdy elementy są blisko swoich docelowych miejsc, oczekuję czasu wykonania bardzo zbliżonego do wariantu optymistycznego, niemal liniowego.
5. **Dane prawie posortowane (globalna wymiana):** Przypadek powinien znaleźć się czasowo pomiędzy danymi losowymi, a prawie posortowanymi (sąsiednia wymiana).



Rysunek 1: Czas wykonania Insertion sort - porównanie scenariuszy danych wejściowych z liniami referencyjnymi $O(n)$ i $O(n^2)$

Rysunek 1 w większości jednoznacznie potwierdza postawione hipotezy. Przypadki skrajne, czyli dane posortowane malejąco (najgorszy przypadek) oraz losowe, rosną równolegle do referencyjnej linii złożoności kwadratowej $O(n^2)$. Z kolei dane ułożone rosnąco (najlepszy przypadek) i te z lokalnymi zaburzeniami (sąsiednia wymiana) wykazują zgodnie z oczekiwaniami czas działania rzędu $O(n)$. Warto jednak zwrócić uwagę na scenariusz danych prawie posortowanych z globalną wymianą. O ile zgodnie z hipotezą jego czas wykonania znajduje się pomiędzy danymi losowymi a wymianą sąsiednią, o tyle nachylenie jego krzywej wyraźnie wskazuje na złożoność $O(n^2)$. Pokazuje to, że szum o charakterze globalnym - nawet gdy jest tak mały jak 10% całości danych - mocno wpływa na działanie Insertion sortu w negatywny sposób.

Czasowa złożoność obliczeniowa

- **Najgorszy i średni przypadek:** $O(n^2)$ - występuje, gdy tablica jest posortowana odwrotnie lub liczby są ułożone losowo. Algorytm musi wtedy dla każdego elementu przechodzić przez większość lub w najgorszym przypadku całą posortowaną już część tablicy.
- **Najlepszy przypadek:** $O(n)$ - zachodzi, gdy tablica jest prawie posortowana gdzie elementy są blisko docelowych miejsc lub całkowicie posortowana. Wewnętrzna pętla szybko lub natychmiast kończy działanie, więc algorytm wykonuje małą ilość lub tylko jedno przejście przez tablicę.

2.2. Selection sort

Zasada działania algorytmu

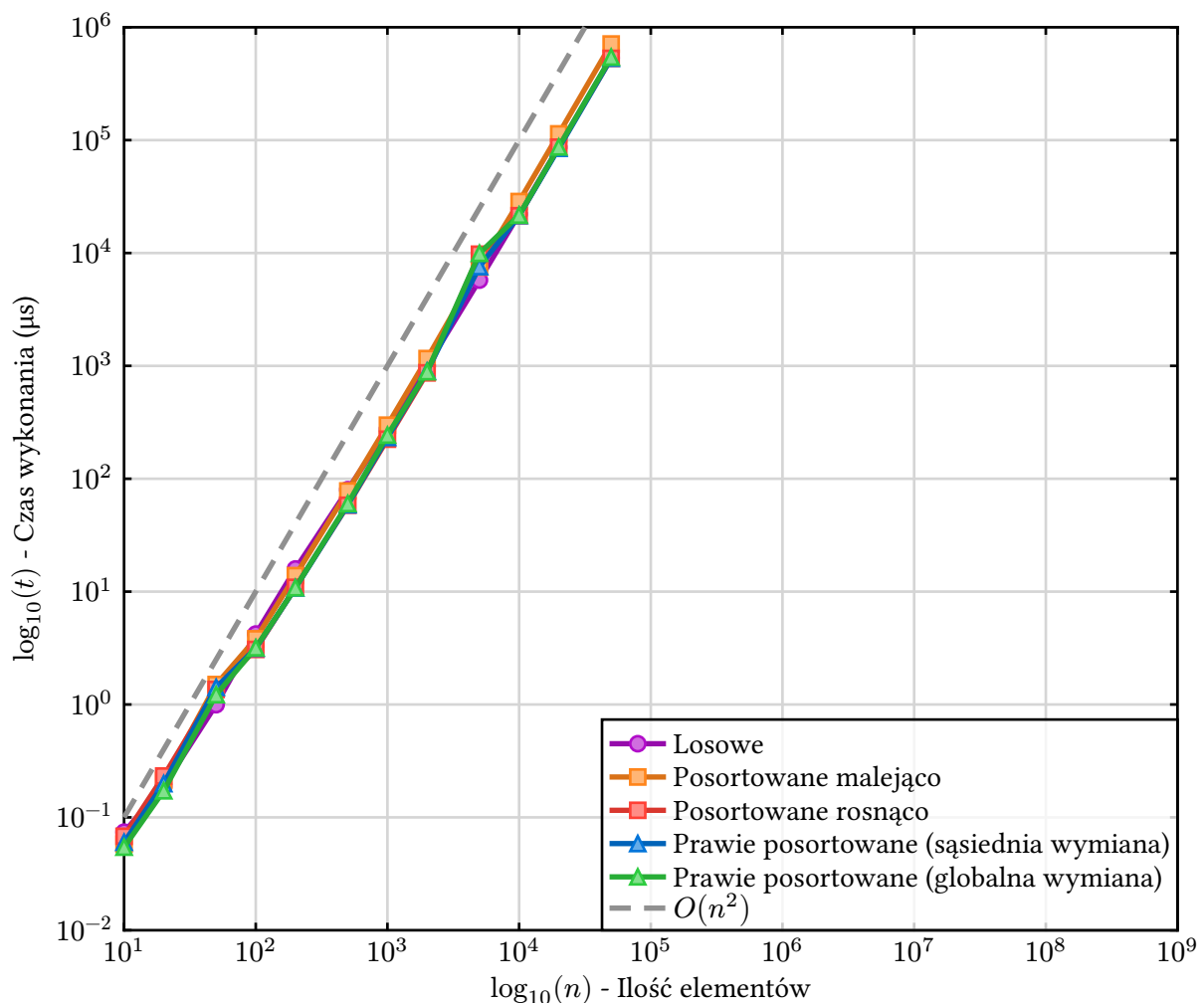
Selection sort polega na wielokrotnym wyszukiwaniu najmniejszego elementu w nieposortowanej części tablicy i zamienianiu go z pierwszym elementem tej nieposortowanej części. Algorytm wirtualnie dzieli tablicę na część posortowaną (z lewej strony) i nieposortowaną (z prawej). W każdym kroku przeszukuje całą prawą stronę, by znaleźć absolutne minimum, a następnie dorzuca je na koniec posortowanej połowy.

Algorytm 2: Selection sort

```
1: procedure SELECTION SORT(arr, n)
2:    $i \leftarrow 0$ 
3:   while  $i < n - 1$  do
4:      $\text{min}_{\text{idx}} \leftarrow i$ 
5:      $j \leftarrow i + 1$ 
6:     while  $j < n$  do
7:       if  $\text{arr}[j] < \text{arr}[\text{min}_{\text{idx}}]$  then
8:          $\text{min}_{\text{idx}} \leftarrow j$ 
9:       end
10:       $j \leftarrow j + 1$ 
11:    end
12:    zamień  $\text{arr}[i]$  z  $\text{arr}[\text{min}_{\text{idx}}]$ 
13:     $i \leftarrow i + 1$ 
14:  end
15: end
```

Hipotezy badawcze dla poszczególnych scenariuszy

1. **Dane losowe:** Oczekuję czasu rzędu $O(n^2)$. Algorytm będzie mało wydajny dla większych tablic.
2. **Dane posortowane malejąco (odwrócone):** Czas działania powinien być identyczny jak dla danych losowych. Liczba porównań pozostaje ta sama, więc nie oczekuję tu drastycznych różnic.
3. **Dane posortowane rosnąco:** Algorytm nie wie, że tablica jest posortowana, więc wykona pełną pulę porównań ($O(n^2)$). Czas może być co najwyżej odrobinę krótszy z powodu braku zamian elementów w pamięci.
4. **Dane prawie posortowane (sąsiednia wymiana):** Ponieważ metoda nie potrafi wykorzystać faktu, że elementy są już blisko swoich miejsc, czas wykonania nie powinien ulec poprawie.
5. **Dane prawie posortowane (globalna wymiana):** Wyniki powinny znów pokrywać się z resztą scenariuszy.



Rysunek 2: Czas wykonania Selection sort - porównanie scenariuszy danych wejściowych z linią referencyjną $O(n^2)$

Rysunek 2 w pełni i jednoznacznie potwierdza postawione hipotezy. Zgodnie z przewidywaniami, wszystkie krzywe na wykresie - niezależnie od tego, czy dane były idealnie posortowane, ułożone malejąco, losowe, czy zawierały lokalne bądź globalne zaburzenia - praktycznie nakładają się na siebie. Rosną one równoległe do szarej linii referencyjnej $O(n^2)$. Wykres ten jest doskonałym dowodem na to, że Selection sort to algorytm całkowicie obojętny na początkowy układ danych wejściowych. Nie widać tu żadnego podziału na najlepszy czy najgorszy przypadek, ponieważ algorytm w każdej sytuacji musi wykonać tę samą, pełną liczbę porównań. Skutkuje to identycznym czasem działania dla każdego z testowanych scenariuszy.

Czasowa złożoność obliczeniowa

- **Najgorszy, średni i najlepszy przypadek:** $O(n^2)$ - jest to cecha szczególna tego algorytmu. Sortowanie przez selekcję jest całkowicie ślepe na początkowe ułożenie danych. Niezależnie od tego, czy tablica jest już idealnie posortowana, czy odwrócona, algorytm i tak musi za każdym razem przeiterować przez resztę tablicy, żeby upewnić się, że znalazł najmniejszą wartość. Zawsze wykonuje dokładnie tę samą liczbę porównań elementów.

2.3. Bubble sort

Zasada działania algorytmu

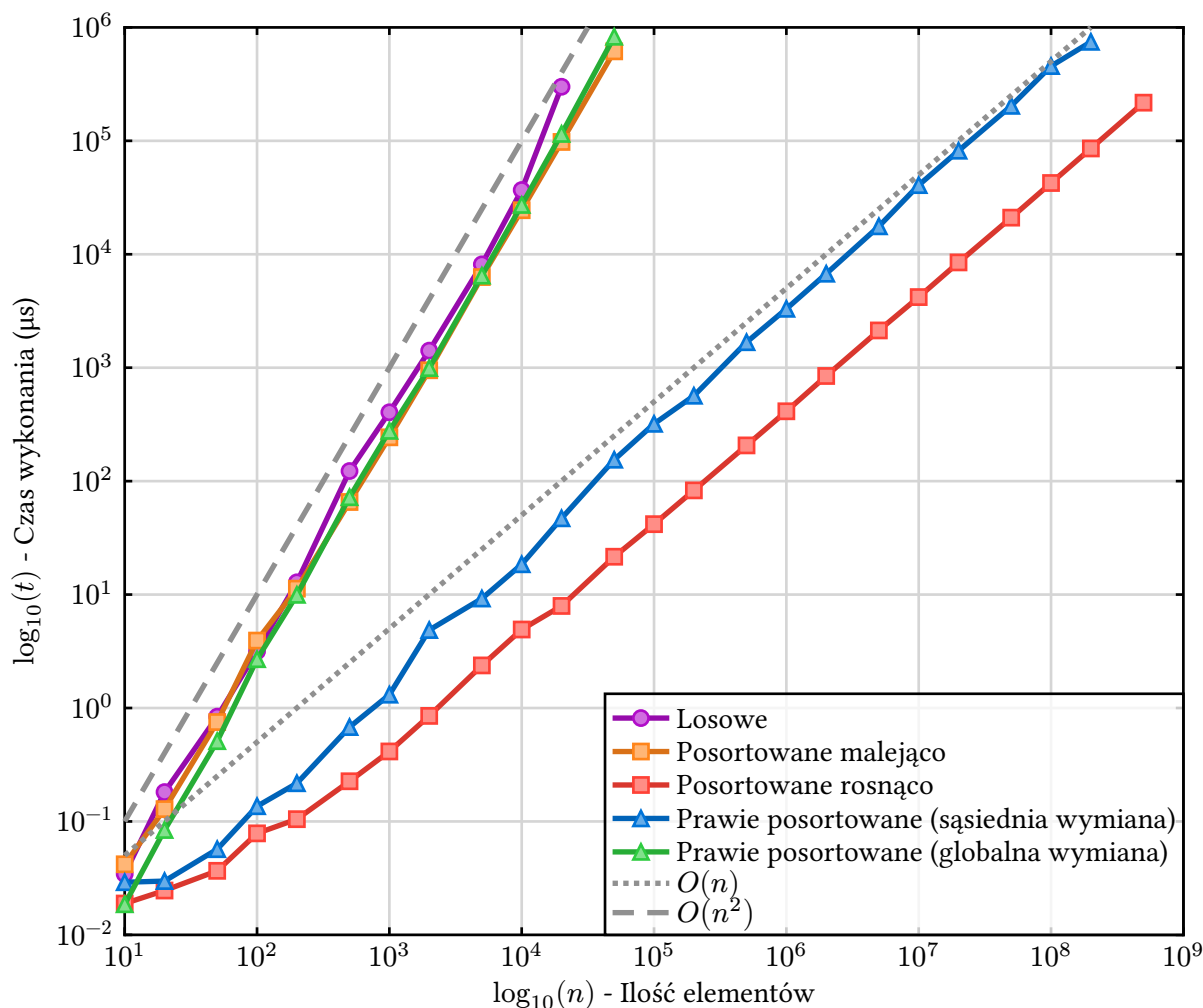
Bubble sort opiera się na wielokrotnym przechodzeniu przez listę i porównywaniu sąsiadujących ze sobą par elementów. Jeśli znajdują się one w niewłaściwej kolejności (pierwszy jest większy od drugiego), są zamieniane miejscami. Po każdym pełnym przejściu, największy z nieposortowanych elementów zostaje przeniesiony na swoją ostateczną pozycję na końcu tablicy - podobnie jak bąbelek powietrza wynurzający się z wody. W pseudokodzie poniżej jest dodatkowa optymalizacja, która powoduje przedwczesne przerwanie algorytmu gdy nie zostanie wykonana żadna zamiana elementów.

Algorytm 3: Bubble sort

```
1: procedure BUBBLE SORT(arr, n)
2:    $i \leftarrow 0$ 
3:   while  $i < n - 1$  do
4:      $j \leftarrow 0$ 
5:      $s \leftarrow 0$ 
6:     while  $j < n - i - 1$  do
7:       if  $\text{arr}[j] > \text{arr}[j + 1]$  then
8:         zamień  $\text{arr}[j]$  z  $\text{arr}[j + 1]$ 
9:          $s \leftarrow 1$ 
10:      end
11:       $j \leftarrow j + 1$ 
12:    end
13:
14:    if  $s = 0$  then
15:      BREAK
16:    end
17:     $i \leftarrow i + 1$ 
18:  end
19: end
```

Hipotezy badawcze dla poszczególnych scenariuszy

1. **Dane losowe:** Oczekuję stosunkowo długi czas działania przez wysoką liczbę zamian, spodziewam się na wykresie $O(n^2)$.
2. **Dane posortowane malejąco (odwrócone):** worst-case - Powoduje konieczność wykonania maksymalnej możliwej liczby zamian, czas wzrośnie drastycznie.
3. **Dane posortowane rosnąco:** best-case - Dzięki fładze sprawdzającej ilość zmian algorytm zrobi tylko $O(n)$ operacji.
4. **Dane prawie posortowane (sąsiednia wymiana):** Dzięki fładze algorytm będzie w stanie skończyć pracę po kilku przejściach. Czas wykonania powinien być znacznie bliższy $O(n)$ niż $O(n^2)$, zbliżony do scenariusza z danymi posortowanymi rosnąco.
5. **Dane prawie posortowane (globalna wymiana):** Losowe zamiany elementów odległych od siebie powodują, że bąbelek musi kilkakrotnie przemierzać tablicę, by przenieść element na właściwe miejsce. Oczekuję czasu gorszego niż przy wymianach sąsiednich, ale nadal zauważalnie lepszego niż dla danych czysto losowych.



Rysunek 3: Czas wykonania Bubble sort - porównanie scenariuszy danych wejściowych z liniami referencyjnymi $O(n)$ i $O(n^2)$

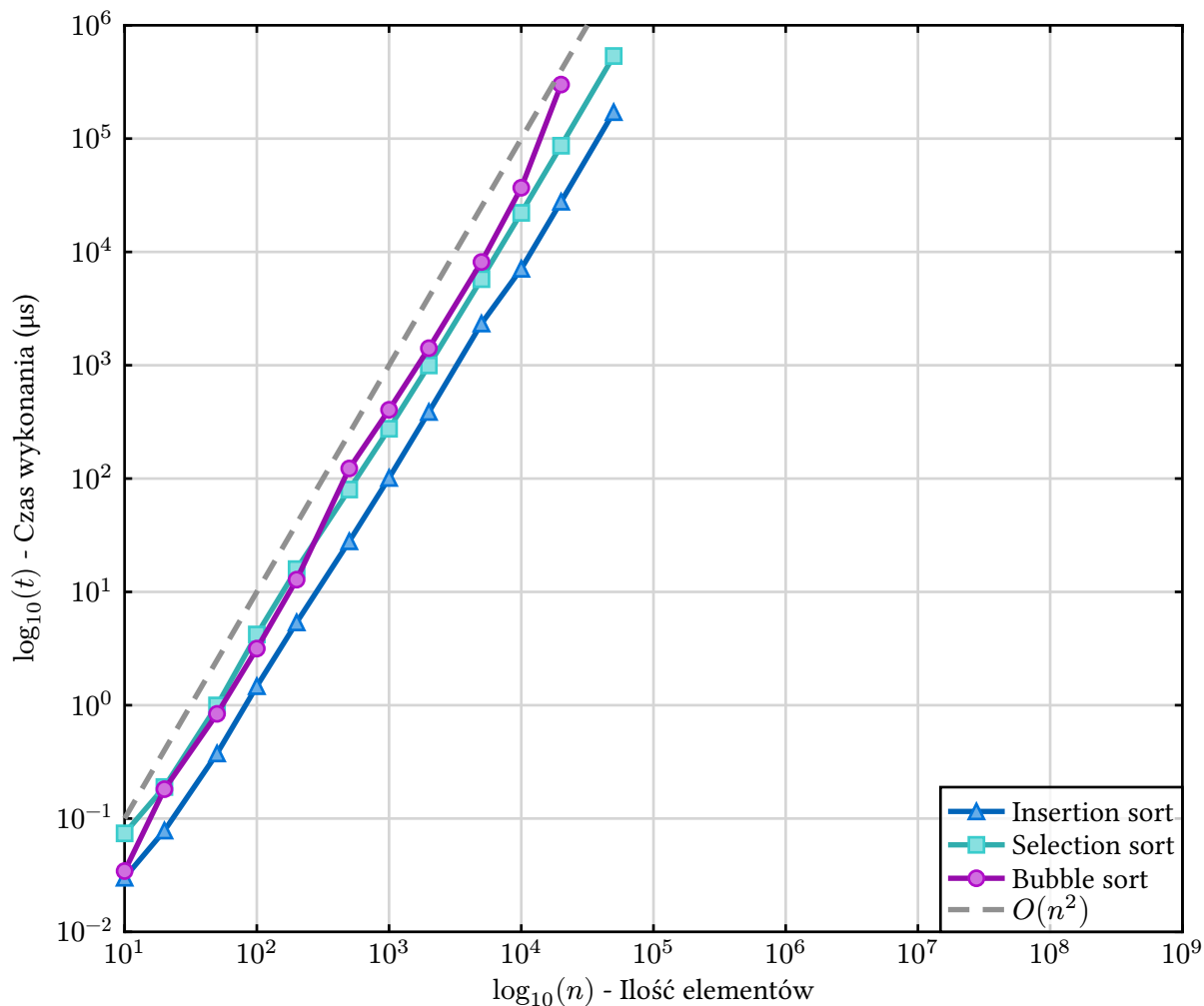
Rysunek 3 w większości potwierdza postawione hipotezy. Przypadki skrajne, czyli dane posortowane malejąco (najgorszy przypadek) oraz losowe, rosną równoległe do referencyjnej linii złożoności kwadratowej $O(n^2)$. Z kolei dane ułożone rosnąco (najlepszy przypadek) i te z lokalnymi zaburzeniami (sąsiednia wymiana) wykazują zgodnie z oczekiwaniami czas działania rzędu $O(n)$, co udowadnia skuteczność optymalizacji z użyciem flagi. Warto jednak zwrócić uwagę na scenariusz danych prawie posortowanych z globalną wymianą. Hipoteza zakładała, że czas ten będzie zauważalnie lepszy niż dla danych losowych. Wykres jednak temu przeczy - zielona krzywa wyraźnie wskazuje na złożoność $O(n^2)$ i nakłada się na najgorsze scenariusze. Pokazuje to, że dalekie zamiany elementów zmuszają algorytm do wielokrotnego przechodzenia przez całą długość tablicy.

Czasowa złożoność obliczeniowa

- **Najgorszy i średni przypadek:** $O(n^2)$ - Wynika to wprost z użycia zagnieżdżonych pętli.
- **Najlepszy przypadek:** $O(n)$ - Zachodzi, gdy tablica jest prawie posortowana gdzie elementy są blisko docelowych miejsc lub całkowicie posortowana. Algorytm wykonuje małą ilość lub tylko jedno przejście przez tablicę.

2.4. Porównanie algorytmów prostszych

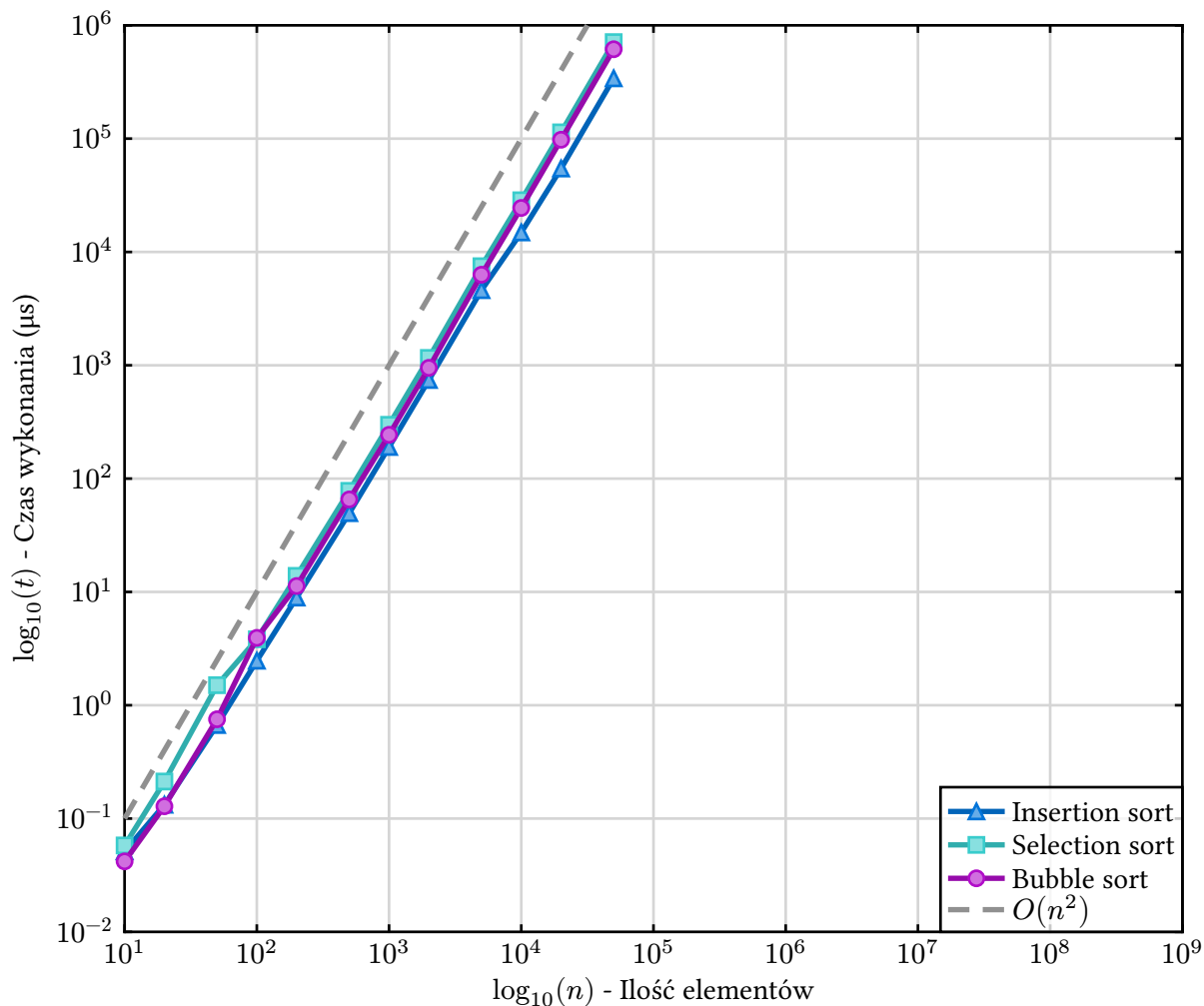
2.4.1. Dane losowe



Rysunek 4: Porównanie algorytmów prostych - dane losowe z linią referencyjną $O(n^2)$

Dla danych losowych najlepiej wypada Insertion sort, osiągając najkrótsze czasy wykonania spośród badanych metod prostych. Na drugim miejscu znajduje się Selection sort, natomiast najwolniejszy w tym zestawieniu okazał się Bubble sort. Jak widać, wszystkie te algorytmy niestety mają złożoność $O(n^2)$ dla danych losowych.

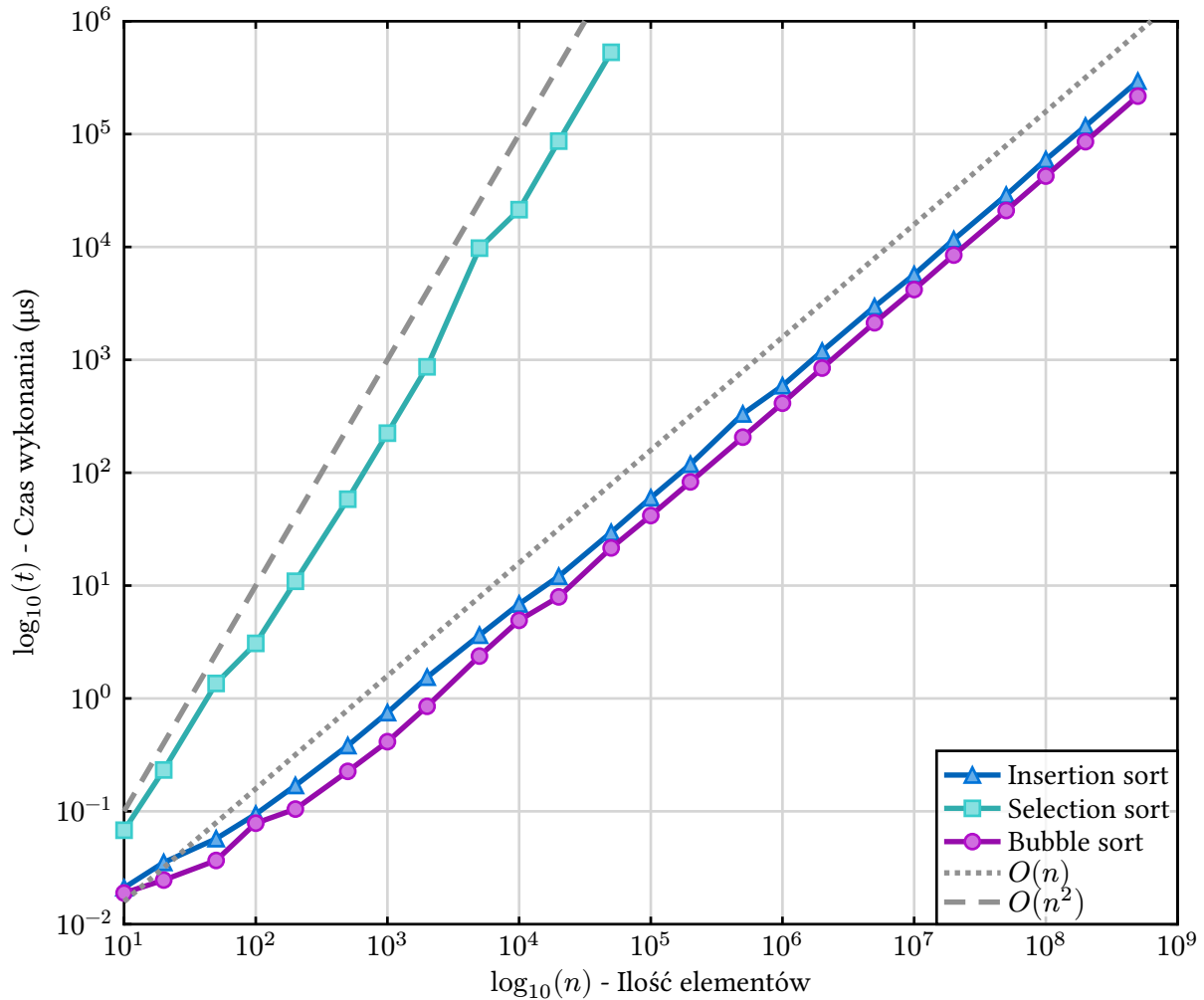
2.4.2. Dane posortowane malejąco (odwrócone)



Rysunek 5: Porównanie algorytmów prostych - dane posortowane malejąco (odwrócone) z linią referencyjną $O(n^2)$

Sytuacja na tym wykresie jest bardzo podobna do scenariusza z danymi losowymi. Ponownie najlepiej wypada Insertion sort, osiągając najkrótszy czas wykonania. Zmieniła się tylko kolejność dwóch pozostałych metod - tym razem to Bubble sort jest minimalnie szybszy od Selection sort, który tutaj okazał się najwolniejszy. Podobnie jak wcześniej, wszystkie te algorytmy mają dla odwróconych danych złożoność $O(n^2)$.

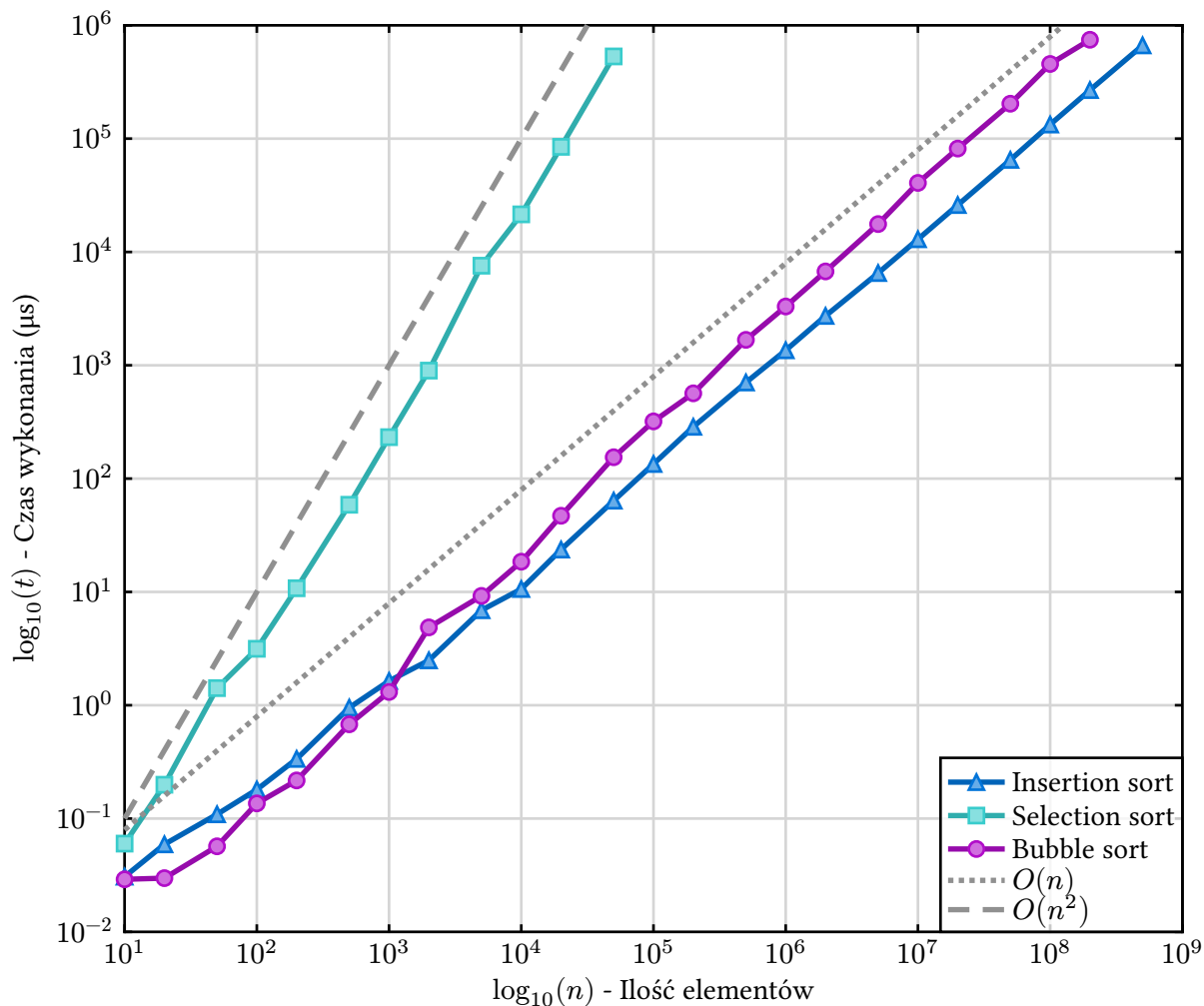
2.4.3. Dane posortowane rosnąco



Rysunek 6: Porównanie algorytmów prostych - dane posortowane rosnąco z liniami referencyjnymi $O(n)$ i $O(n^2)$

W przypadku danych posortowanych rosnąco widzimy zupełnie inną sytuację niż na poprzednich wykresach. Tym razem najszybszy okazuje się Bubble sort, choć Insertion sort jest tylko minimalnie wolniejszy. Oba te algorytmy świetnie radzą sobie z takim ułożeniem danych, co widać po ich krzywych rosnących wzdłuż linii referencyjnej $O(n)$. Z kolei Selection sort nie potrafi wykorzystać faktu, że tablica jest już ułożona prawidłowo, i jako jedyny pozostaje przy złożoności $O(n^2)$, działając zdecydowanie najgorzej.

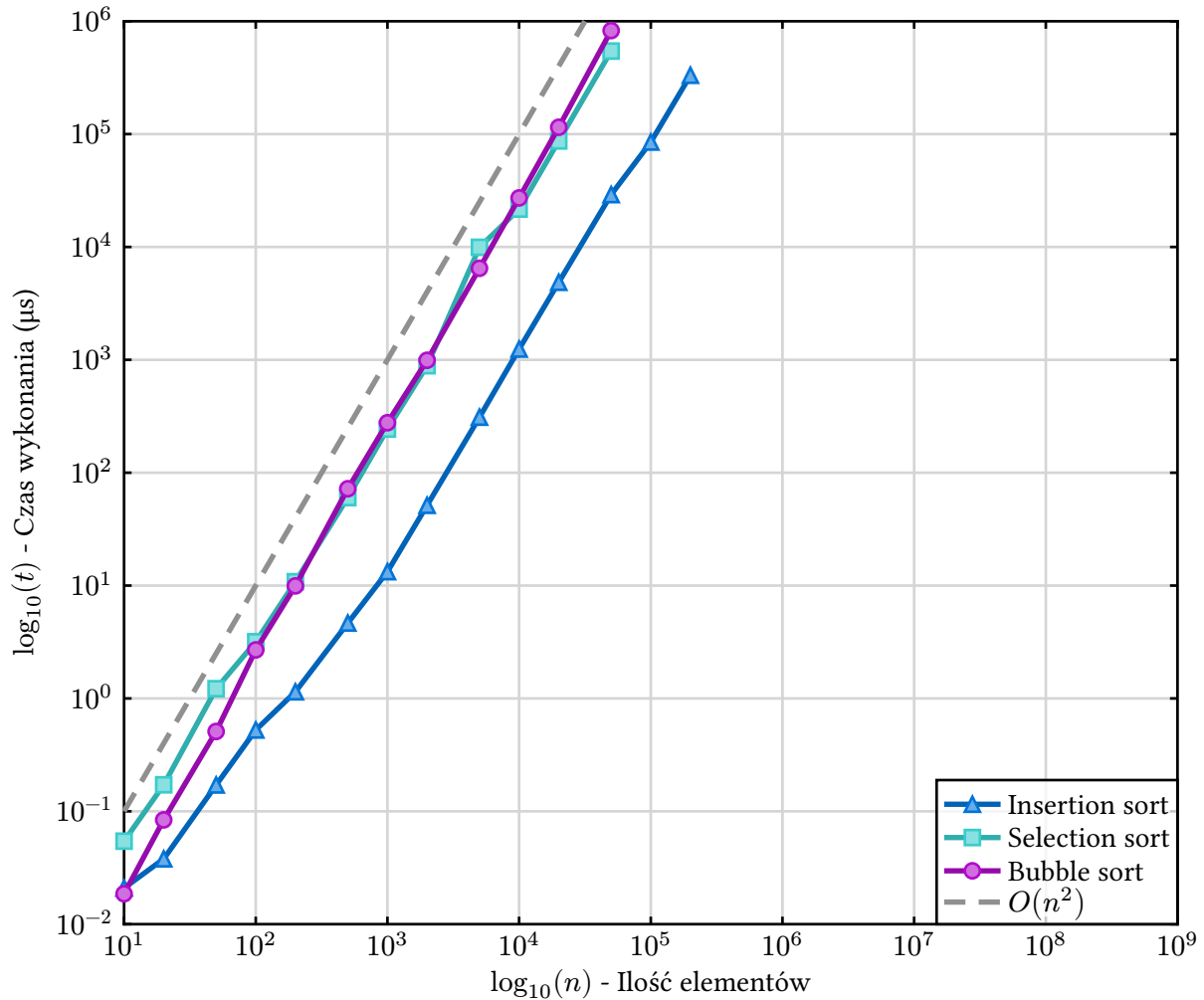
2.4.4. Dane prawie posortowane (sąsiednia wymiana)



Rysunek 7: Porównanie algorytmów prostych - dane prawie posortowane (sąsiednia wymiana) z liniami referencyjnymi $O(n)$ i $O(n^2)$

Dla danych prawie posortowanych z sąsiednią wymianą sytuacja jest zbliżona do wykresu z danymi ułożonymi rosnąco. Najszybszy dla większych zestawów danych okazuje się Insertion sort, chociaż przy bardzo małych tablicach (do 10^3 elementów) Bubble sort działa odrobinę szybciej. Obie te metody bardzo dobrze wykorzystują fakt, że dane są tylko lekko zaburzone i osiągają czas działania wzdłuż linii referencyjnej $O(n)$. Selection sort ponownie nie potrafi zaadaptować się do ułożenia elementów, przez co jest zdecydowanie najwolniejszy i jako jedyny zachowuje złożoność $O(n^2)$.

2.4.5. Dane prawie posortowane (globalna wymiana)



Rysunek 8: Porównanie algorytmów prostych - dane prawie posortowane (globalna wymiana) z linią referencyjną $O(n^2)$

W scenariuszu z danymi prawie posortowanymi z globalną wymianą zdecydowanie najlepiej wypada Insertion sort, osiągając najkrótszy czas wykonania. Selection sort i Bubble sort są znacznie wolniejsze i ich krzywe niemal się pokrywają, chociaż to Bubble sort jest minimalnie najwolniejszy przy większych rozmiarach tablic. Wykres pokazuje również, że globalne zaburzenia w danych powodują, że wszystkie trzy algorytmy działają w tym przypadku ze złożonością $O(n^2)$.

3. Algorytmy efektywniejsze

3.1. Quicksort

Zasada działania algorytmu

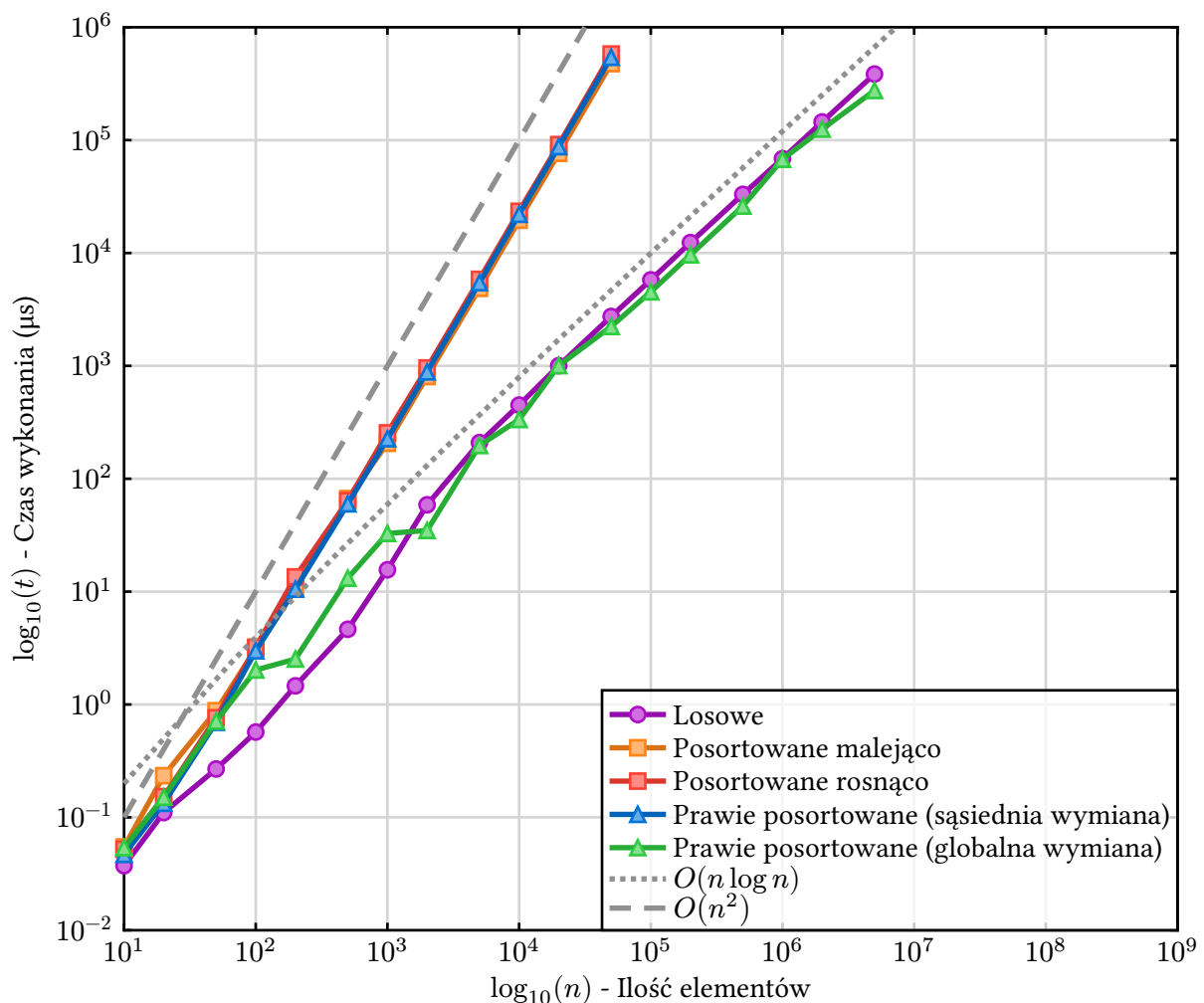
Quicksort to klasyczny algorytm oparty na strategii „dziel i zwyciężaj”. Wybierany jest element nazywany pivotem (w poniższej implementacji jest to zawsze ostatni element zakresu), a następnie tablica jest przestawiana tak, by wszystkie elementy mniejsze od pivota znalazły się po jego lewej stronie, a większe po prawej. Tę operację nazywamy partycjonowaniem. Następnie algorytm rekurencyjnie wywołuje siebie dla lewej i prawej podtablicy. Podział ten trwa aż do momentu, gdy podtablice są jednoelementowe i z definicji posortowane.

Algorytm 4: Quicksort

```
1: function PARTITION(arr, l, r)
2:    $i \leftarrow l$ 
3:    $j \leftarrow r$ 
4:   while  $j < r$  do
5:     if  $\text{arr}[j] < \text{arr}[r]$  then
6:       zamień  $\text{arr}[i]$  z  $\text{arr}[j]$ 
7:        $i \leftarrow i + 1$ 
8:     end
9:      $j \leftarrow j - 1$ 
10:  end
11:  zamień  $\text{arr}[i]$  z  $\text{arr}[r]$ 
12:  return  $i$ 
13: end
14:
15: procedure QUICKSORT(arr, l, r)
16:  if  $l < r$  then
17:     $p \leftarrow \text{PARTITION}(\text{arr}, l, r)$ 
18:    QUICKSORT(arr, l,  $p - 1$ )
19:    QUICKSORT(arr,  $p + 1$ , r)
20:  end
21: end
```

Hipotezy badawcze dla poszczególnych scenariuszy

1. **Dane losowe:** Oczekuję zachowania bliskiego $O(n \log n)$, jako, że za każdym razem powinniśmy dzielić problem na pół. Pivot wybierany losowo z perspektywy wartości rzadko będzie skrajny, więc drzewo rekursji powinno być płytkie i zrównoważone.
2. **Dane posortowane malejąco (odwrócone):** worst-case - Ostatni element jest zawsze minimum zakresu, więc partycjonowanie tworzy skrajnie nie zrównoważone podziały (jeden element po lewej, reszta po prawej). Czas powinien wzrosnąć do $O(n^2)$.
3. **Dane posortowane rosnąco:** worst-case - Analogicznie do scenariusza malejącego, pivot jest zawsze maksimum zakresu, będziemy mieli $O(n^2)$. Spodziewam się czasu zbliżonego do danych malejących, oba znacznie wolniejsze od danych losowych.
4. **Dane prawie posortowane (sąsiednia wymiana):** Nieliczne zamiany sąsiednich elementów nie powinny wywołać dużego skoku w losowości tablicy, spodziewam się, że pivot dość często będzie skrajnym elementem. Oczekuję czasu $O(n^2)$.
5. **Dane prawie posortowane (globalna wymiana):** Losowe zamiany odległych elementów skuteczniej dezorganizują tablicę niż zamiany sąsiednie, dzięki czemu pivot powinien być lepiej dobrany. Spodziewam się wyniku wyraźnie lepszego niż przy wymianach sąsiednich, zbliżonego do danych losowych.



Rysunek 9: Czas wykonania Quicksort - porównanie scenariuszy danych wejściowych z liniami referencyjnymi $O(n \log n)$ i $O(n^2)$

Rysunek 9 jednoznacznie potwierdza wszystkie postawione hipotezy. Scenariusze, w których dane były początkowo posortowane rosnąco, malejąco, a także te z drobnymi zaburzeniami (sąsiednia wymiana), rosną równolegle do referencyjnej linii złożoności kwadratowej $O(n^2)$. Pokazuje to wyraźnie, że wybieranie ostatniego elementu jako pivota przy takim ułożeniu wartości prowadzi do najgorszego możliwego wariantu i skrajnie nierównych podziałów. Z kolei dane ułożone losowo oraz te z globalną wymianą wykazują zgodnie z oczekiwaniami czas działania rzędu $O(n \log n)$. Potwierdza to, że całkowicie losowe ułożenie elementów lub wystarczająco duże zaburzenia w postaci dalekich zamian pozwalają algorytmowi na w miarę równomierne dzielenie problemu, co drastycznie poprawia jego wydajność względem uporządkowanych tablic.

Czasowa złożoność obliczeniowa

- **Najgorszy przypadek:** $O(n^2)$ - Zachodzi, gdy pivot za każdym razem trafia na skrajną pozycję (np. jest minimalnym lub maksymalnym elementem zakresu). Przy zastosowanej strategii wyboru ostatniego elementu jako pivota, dokładnie taki scenariusz wystąpi dla tablic już posortowanych rosnąco lub malejąco.
- **Średni i najlepszy przypadek:** $O(n \log n)$ - Gdy pivot dzieli tablicę na w miarę równe części, głębokość rekursji wynosi $O(\log n)$, a każdy poziom wymaga liniowej pracy.

3.2. Shellsort

Zasada działania algorytmu

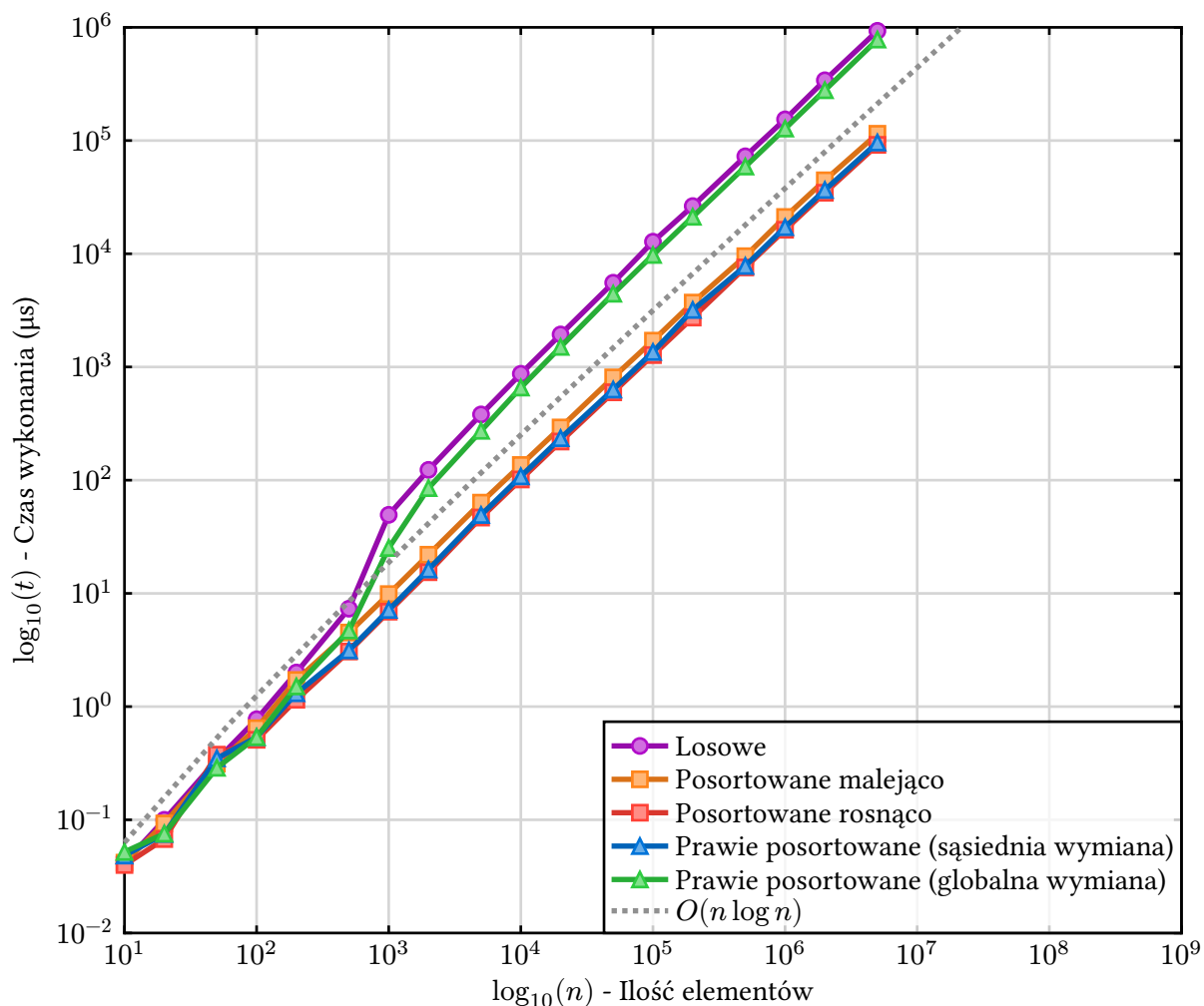
Shellsort jest uogólnieniem Insertion sorta. Zamiast porównywać i wstawiać sąsiadujące elementy, algorytm operuje na elementach oddalonych o pewien krok (gap). Początkowo krok jest duży (w tej implementacji $\frac{n}{2}$), co pozwala szybko przenosić elementy daleko od ich docelowych pozycji. Następnie krok jest stopniowo zmniejszany o połowę, aż wyniesie 1 - wtedy algorytm staje się zwykłym Insertion sortem, jednak działa on na danych, które są już prawie posortowane po poprzednich przejściach, dzięki czemu wewnętrzna pętla wykonuje bardzo mało przesunięć.

Algorytm 5: Shellsort

```
1: procedure SHELLSORT(arr, n)
2:   gap  $\leftarrow \frac{n}{2}$ 
3:   while gap > 0 do
4:     i  $\leftarrow$  gap
5:     while i < n do
6:       temp  $\leftarrow$  arr[i]
7:       j  $\leftarrow$  i
8:       while j  $\geq$  gap i arr[j - gap] > temp do
9:         arr[j]  $\leftarrow$  arr[j - gap]
10:        j  $\leftarrow$  j - gap
11:      end
12:      arr[j]  $\leftarrow$  temp
13:      i  $\leftarrow$  i + 1
14:    end
15:    gap  $\leftarrow \frac{gap}{2}$ 
16:  end
17: end
```

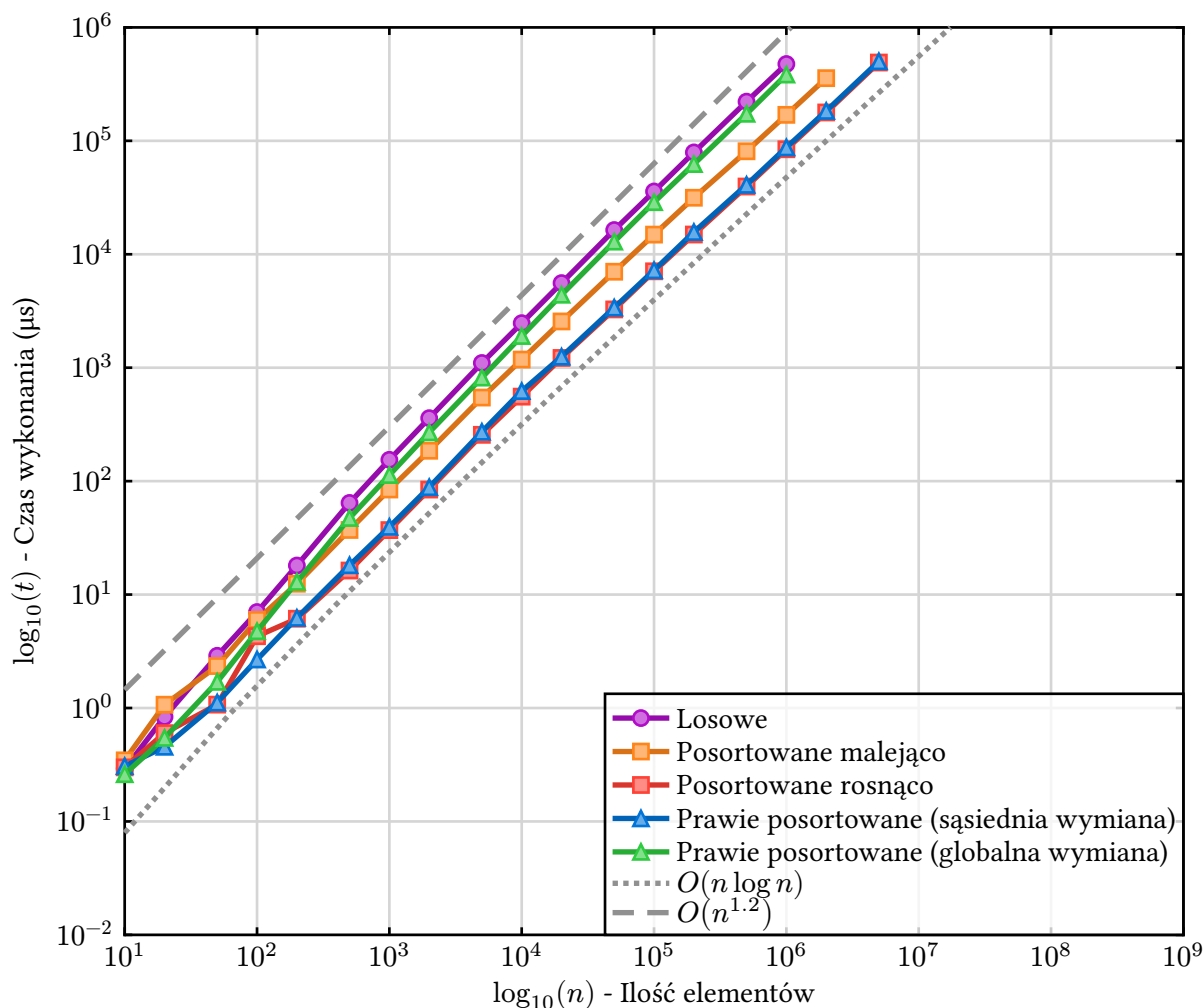
Hipotezy badawcze dla poszczególnych scenariuszy

1. **Dane losowe:** Oczekuję wyraźnie lepszego czasu niż proste algorytmy $O(n^2)$. Duże początkowe kroki szybko redukują nieporządek w tablicy, dzięki czemu ostatni przebieg z krokiem 1 jest niemal natychmiastowy.
2. **Dane posortowane malejąco (odwrócone):** Algorytm powinien dobrze sobie poradzić, bo pierwsze duże kroki przeniosą elementy z początku tablicy na koniec. Spodziewam się czasu minimalnie wolniejszego od już posortowanej tablicy przez potrzebę zamiany elementów w tablicy.
3. **Dane posortowane rosnąco:** best-case — Przy każdym kroku wewnętrzna pętla nie wykonuje żadnych przesunięć, bo elementy odległe o gap są już we właściwej kolejności. Powinniśmy zobaczyć na wykresie $O(n \log n)$.
4. **Dane prawie posortowane (sąsiednia wymiana):** Bardzo bliskie scenariuszowi rosnącemu. Nieliczne zaburzenia powodują znikome dodatkowe przedstawienia przy małych krokach. Spodziewam się czasu praktycznie identycznego z wariantem posortowanym rosnąco.
5. **Dane prawie posortowane (globalna wymiana):** Elementy oddalone od swoich docelowych pozycji są szybko przemieszczane przez duże kroki. Oczekuję wyników bardzo zbliżonych do danych losowych, nieznacznie lepszych dzięki ogólnie mniejszemu nieporządkowi.



Rysunek 10: Czas wykonania Shellsort - porównanie scenariuszy danych wejściowych z linią referencyjną $O(n \log n)$

Zanim przejdziemy do analizy postawionych wcześniej hipotez musimy zauważyć, że przy 10^3 elementów można zaobserwować skok w długości czasu wykonania dla danych losowych i prawie posortowanych z globalną wymianą elementów. Od tego momentu, algorytm zaczyna działać znacznie wolniej. Nie udało mi się znaleźć dokładnej przyczyny tego zjawiska, ale wnioskuję, że nie jest to problem z cachem, jako, że mały rozmiar tablicy nie jest w stanie go przepełnić, gdyby była to wina cachea to spodziewał bym się takiego problemu bliżej 10^4 elementów tablicy. Rozwiązanie tej zagadki robi się jednak nieco jaśniejsze gdy testy wykonamy przy optymalizacji -00 tak jak na Rysunek 11.



Rysunek 11: Czas wykonania Shellsort skompilowanego z flagą -O0 - porównanie scenariuszy danych wejściowych z liniami referencyjnymi $O(n \log n)$ i $O(n^{1.2})$

Na Rysunek 11 widzimy, że nagły skok w czasie wykonywania znika, można więc wywnioskować, że powodem skoku była jakaś optymalizacja, którą kompilator mógł wdrożyć dla tablic o rozmiarze mniejszym niż 10^3 w przypadku Shellsorta. Rysunek 10 w większości potwierdza postawione hipotezy. Scenariusze dla danych posortowanych rosnąco, malejąco oraz z sąsiednią wymianą osiągają bardzo zbliżone, najlepsze wyniki, rosnąc równoległe do linii referencyjnej $O(n \log n)$. Zgodnie z założeniami, konieczność odwrócenia całej tablicy (dane malejące) tylko nieznacznie wydłuża czas pracy w porównaniu do danych już posortowanych. Jednak na pewno nie spodziewałem się nagłego skoku w czasie wykonywania algorytmu dla danych losowych i prawie posortowanych z globalną wymianą. Pokazuje to, jak ważne jest testowanie algorytmów i porównywanie ich w sposób empiryczny.

Czasowa złożoność obliczeniowa

- **Najgorszy przypadek:** $O(n^2)$ - Przy oryginalnej sekwencji Shella (dzielenie przez 2). Istnieją sekwencje kroków (np. Hibbarda czy Pratt), które gwarantują lepszy wynik, jednak nie są tu stosowane.
- **Średni przypadek:** W praktyce zwykle $O(n^{\frac{3}{2}})$ lub lepiej, zdecydowanie szybszy niż zwykłe algorytmy $O(n^2)$.
- **Najlepszy przypadek:** $O(n \log n)$ - Gdy dane są już niemal posortowane, większość przejść nie wykonuje żadnych przestawień.

3.3. Heapsort

Zasada działania algorytmu

Heapsort działa dwuetapowo. W pierwszym etapie tablica jest przekształcana w strukturę zwaną kopcem maksymalnym (max-heap), czyli drzewo binarne, w którym każdy węzeł jest większy od swoich dzieci. Gwarantuje to, że korzeń (pierwszy element tablicy) zawsze zawiera maksimum. W drugim etapie algorytm wielokrotnie zamienia korzeń z ostatnim elementem kopca (odkładając tym samym maksimum na właściwe miejsce na końcu tablicy), a następnie przywraca własność kopca dla zmniejszonej o jeden struktury. Procedura `heapify` realizuje właśnie to przywracanie, opadając w dół drzewa.

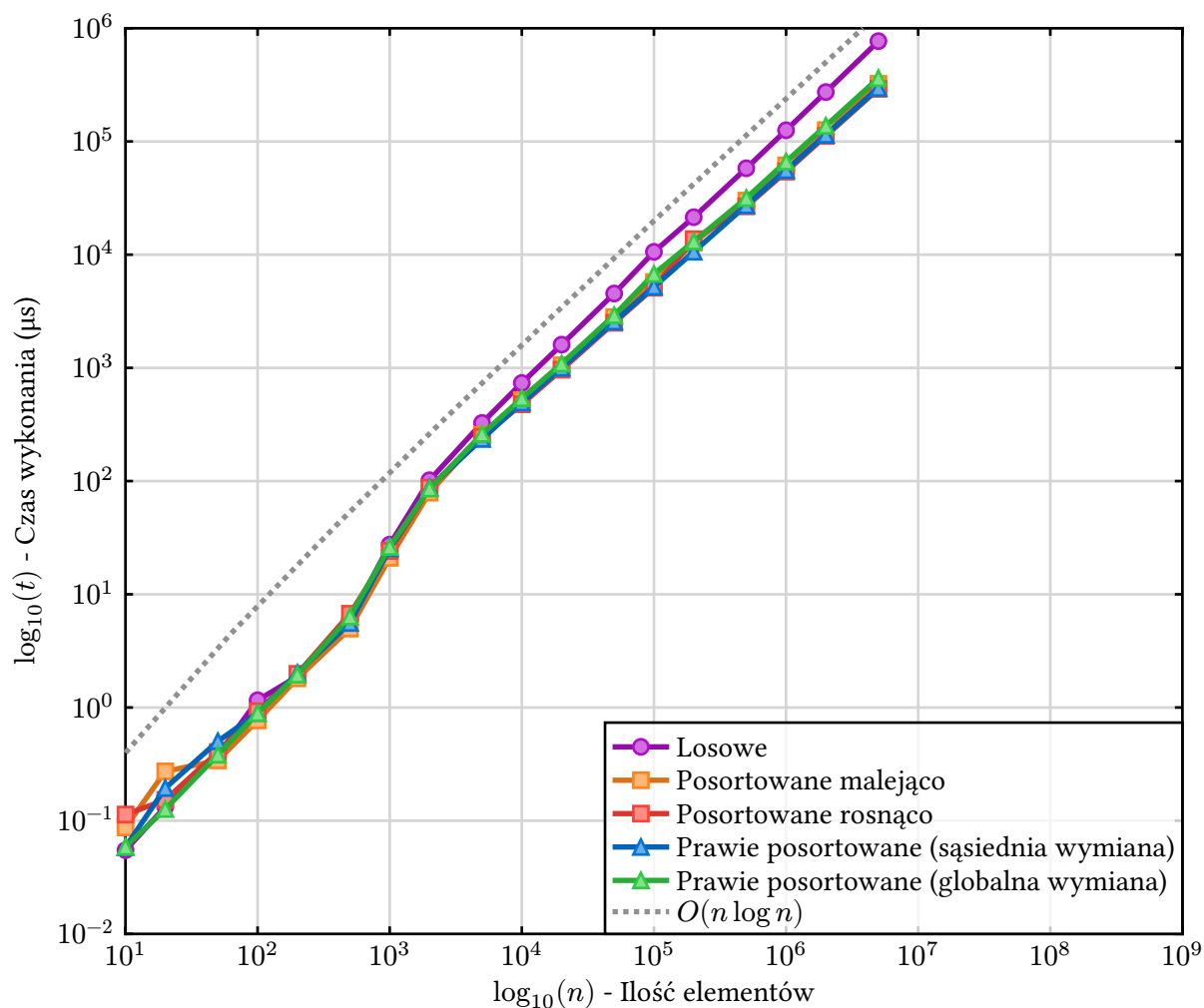
Algorytm 6: Heapsort

```
1: procedure HEAPIFY(arr, n, i)
2:   largest ← i
3:   left ← 2 * i + 1
4:   right ← 2 * i + 2
5:   if left < n i arr[left] > arr[largest] then
6:     largest ← left
7:   end
8:   if right < n i arr[right] > arr[largest] then
9:     largest ← right
10:  end
11:  if largest ≠ i then
12:    zamień arr[i] z arr[largest]
13:    HEAPIFY(arr, n, largest)
14:  end
15: end
16:
17: procedure HEAPSORT(arr, n)
18:   i ←  $\frac{n}{2} - 1$ 
19:   while i ≥ 0 do
20:     HEAPIFY(arr, n, i)
21:     i ← i - 1
22:   end
23:
24:   i ← n - 1
25:   while i > 0 do
26:     zamień arr[0] z arr[i]
27:     HEAPIFY(arr, i, 0)
28:     i ← i - 1
29:   end
30: end
```

Hipotezy badawcze dla poszczególnych scenariuszy

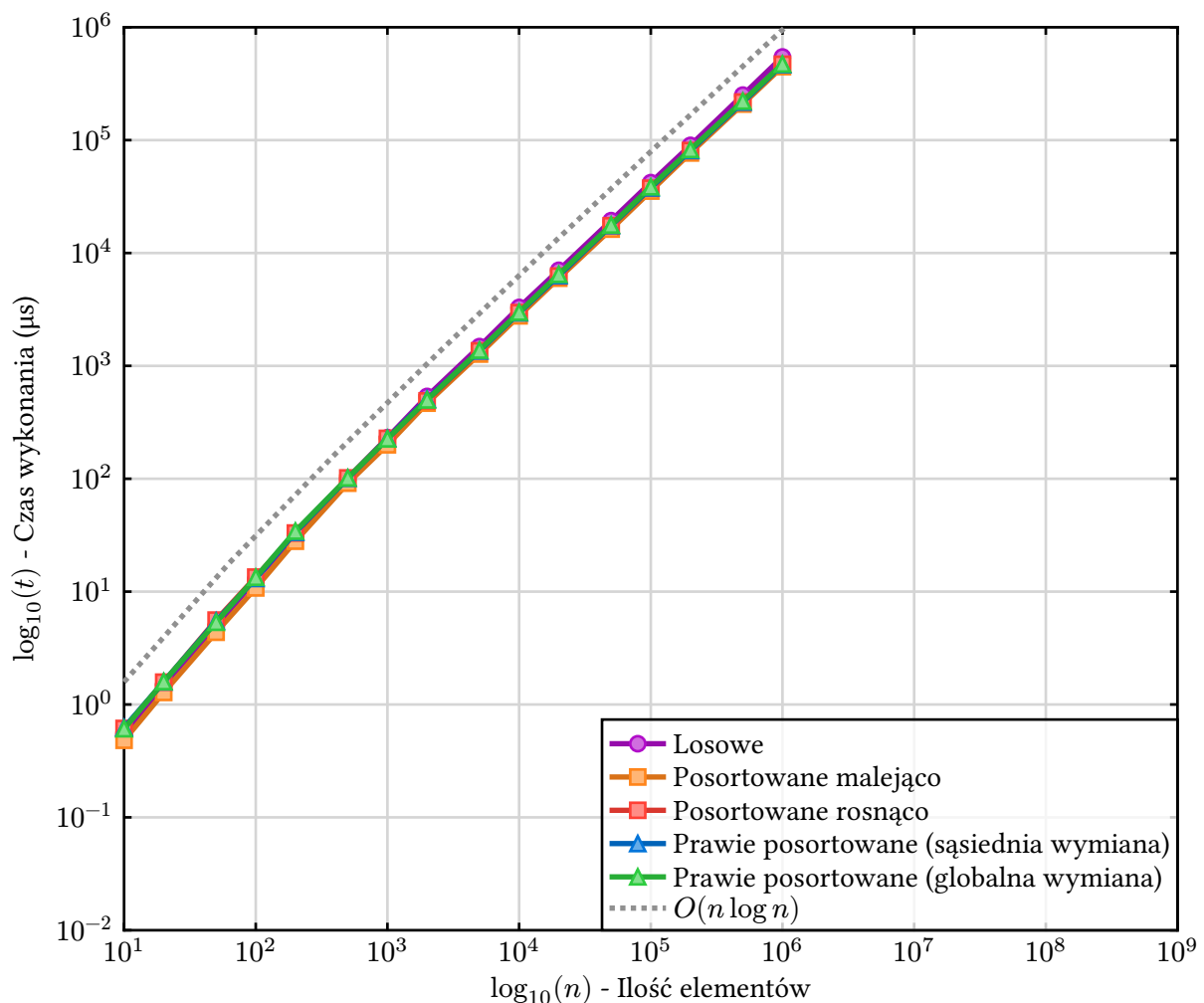
1. **Dane losowe:** Oczekuję stabilnego czasu rzędu $O(n \log n)$.
2. **Dane posortowane malejąco (odwrócone):** W przeciwieństwie do Quicksorta, Heapsort nie ma tutaj problemu. Budowanie kopca z odwróconej tablicy przebiega sprawnie, a dalsze etapy sortowania są identyczne. Oczekuję czasu zbliżonego do danych losowych.
3. **Dane posortowane rosnąco:** Analogicznie, algorytm wykona tę samą pracę niezależnie od wejścia. Dane posortowane rosnąco mogą jednak powodować nieco więcej zamian przy budowaniu kopca, co może skutkować minimalnie gorszym czasem niż dla danych losowych.
4. **Dane prawie posortowane (sąsiednia wymiana):** Algorytm nie potrafi wykorzystać faktu, że dane są niemal posortowane. Oczekuję czasu bardzo zbliżonego do scenariusza z danymi losowymi.

5. **Dane prawie posortowane (globalna wymiana):** Tak samo jak powyżej, Heapsort będzie zachowywał się praktycznie identycznie we wszystkich pięciu scenariuszach.



Rysunek 12: Czas wykonania Heapsort - porównanie scenariuszy danych wejściowych z linią referencyjną $O(n \log n)$

O dziwo Heapsort pokazuje nam bardzo podobny skok w czasie wykonania algorytmu przy 10^3 elementów, ale potencjalny powód tego zachowania już został omówiony. Możemy tutaj za to zauważyć jeszcze inną ciekawą rzecz, czyli powolne odstawanie danych losowych od całej reszty na wykresie. Najpierw zobaczmy jednak wykres Heapsorta z optymalizacją -00, aby upewnić się, że skok rzeczywiście znika.



Rysunek 13: Czas wykonania Heapsort skompilowanego z flagą -O0 - porównanie scenariuszy danych wejściowych z linią referencyjną $O(n \log n)$

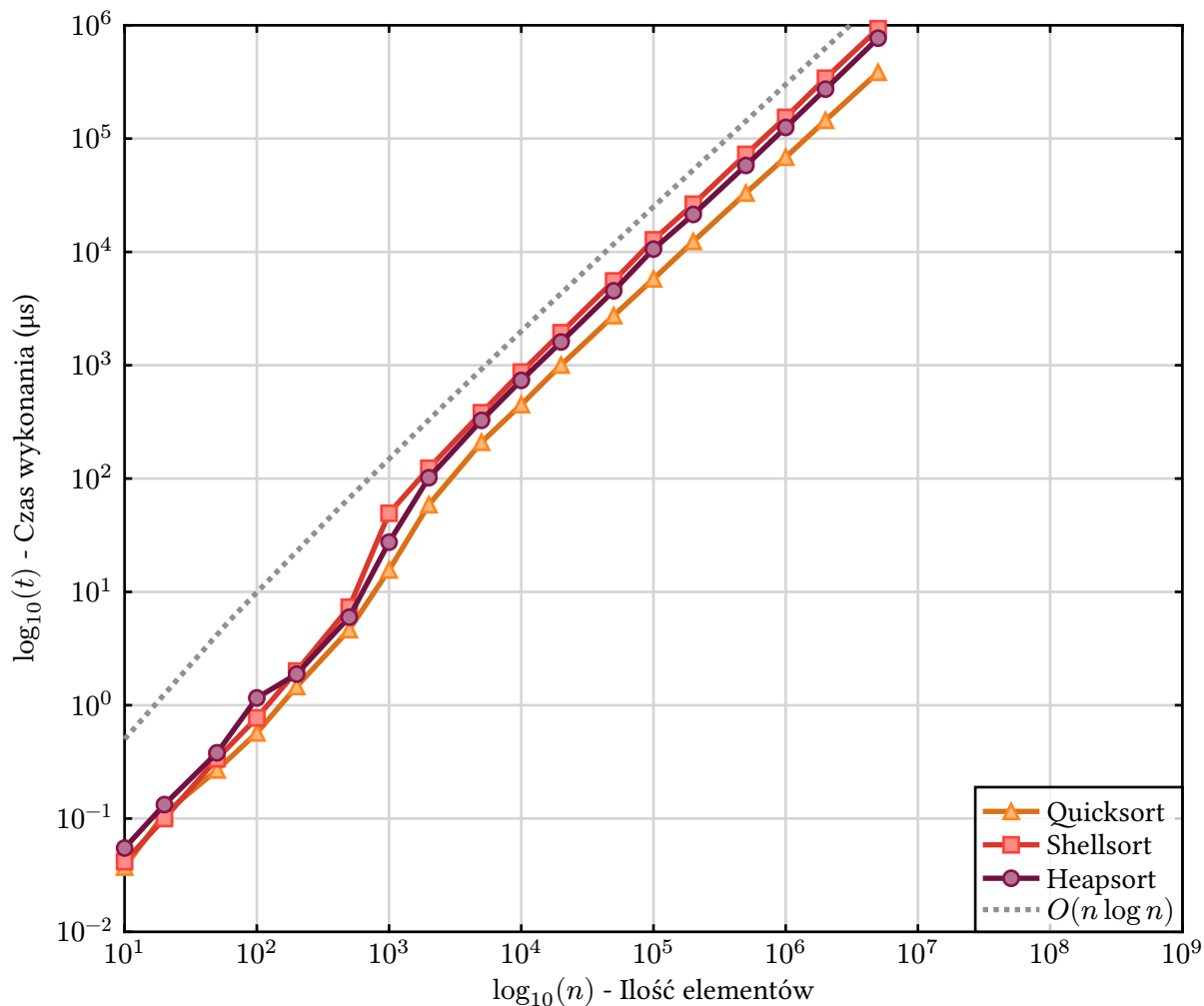
Jak widać na Rysunek 13, skok znika, przejdźmy więc do analizy wcześniej postawionych hipotez. Rysunek 12 w dużej mierze potwierdza postawione hipotezy, udowadniając, że Heapsort nie reaguje drastycznie na początkowy układ danych. Wszystkie krzywe rosną w tym samym tempie, równoległe do linii referencyjnej $O(n \log n)$, co pokazuje stałą złożoność czasową algorytmu niezależnie od scenariusza. Wykres ujawnia jednak jedno ciekawe odstępstwo od początkowych założeń. Zakładaliśmy, że wszystkie warianty będą miały bardzo zbliżony czas, a dane posortowane rosnąco mogą wypaść minimalnie gorzej przez budowanie kopca. W rzeczywistości to dane całkowicie losowe działają zauważalnie wolniej (fioletowa krzywa wyraźnie oddziela się od reszty i znajduje się najwyżej), podczas gdy pozostałe cztery przypadki praktycznie się pokrywają. Wynika to prawdopodobnie z optymalizacji na jakie pozwala -O3 dla Heapsorta w przypadkach z bardziej przewidywalnymi danymi.

Czasowa złożoność obliczeniowa

- **Najgorszy, średni i najlepszy przypadek:** $O(n \log n)$ - Budowanie kopca kosztuje $O(n)$, a każde z n wywołań `heapify` podczas wyciągania elementów kosztuje $O(\log n)$. Co istotne, Heapsort jest algorytmem nieadaptywnym - jego złożoność nie zmienia się w zależności od ułożenia danych wejściowych.

3.4. Porównanie algorytmów efektywniejszych

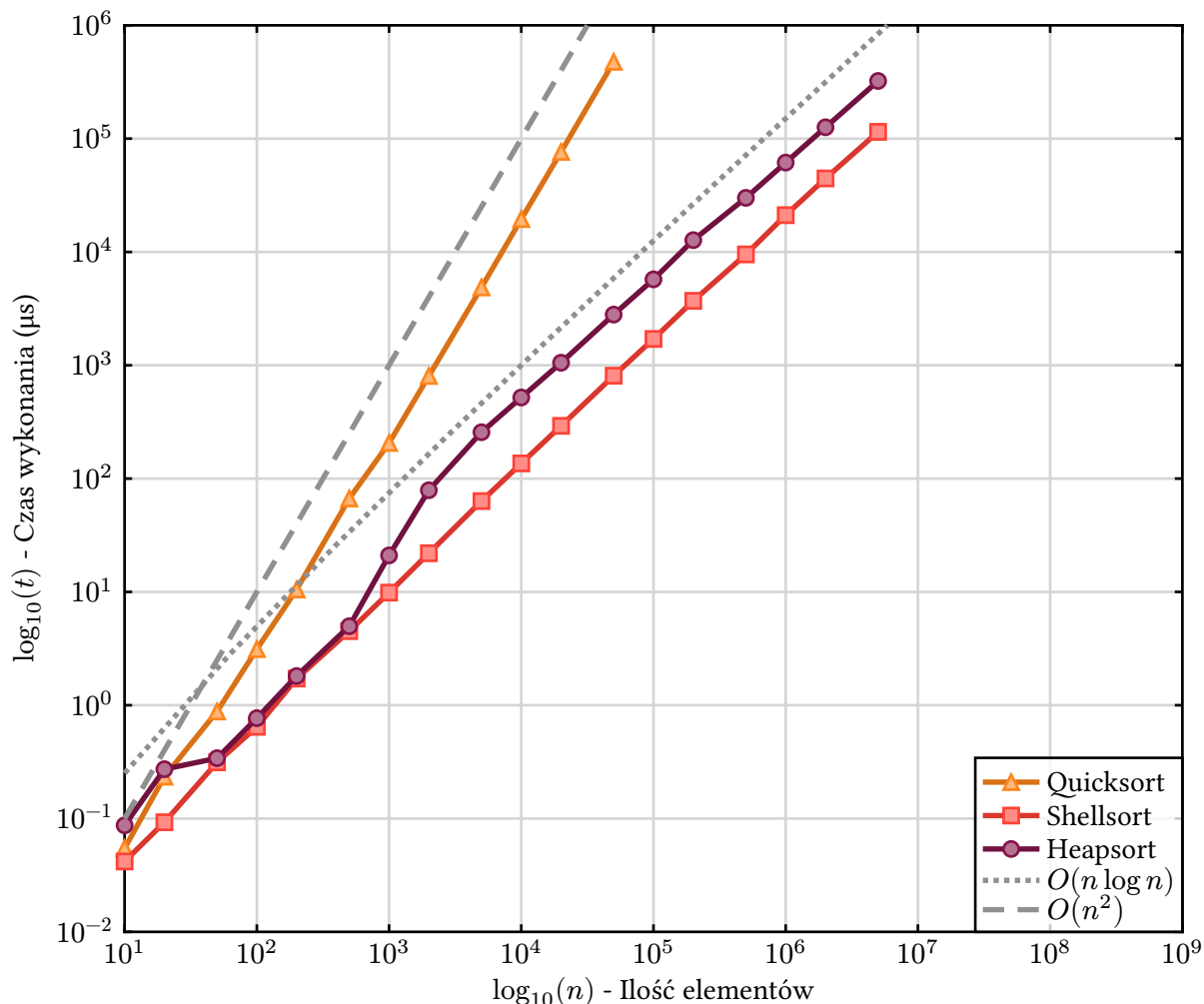
3.4.1. Dane losowe



Rysunek 14: Porównanie algorytmów efektywniejszych - dane losowe z linią referencyjną $O(n \log n)$

W scenariuszu dla danych losowych najlepiej wypada Quicksort, osiągając najkrótsze czasy wykonania spośród badanych algorytmów efektywniejszych. Na drugim miejscu znajduje się Heapsort, natomiast najwolniejszy w tym zestawieniu okazał się Shellsort. Zgodnie z wykresem, krzywe wszystkich trzech algorytmów rosną równoległe do linii referencyjnej $O(n \log n)$, co w praktyce potwierdza ich złożoność czasową dla losowo ułożonych elementów.

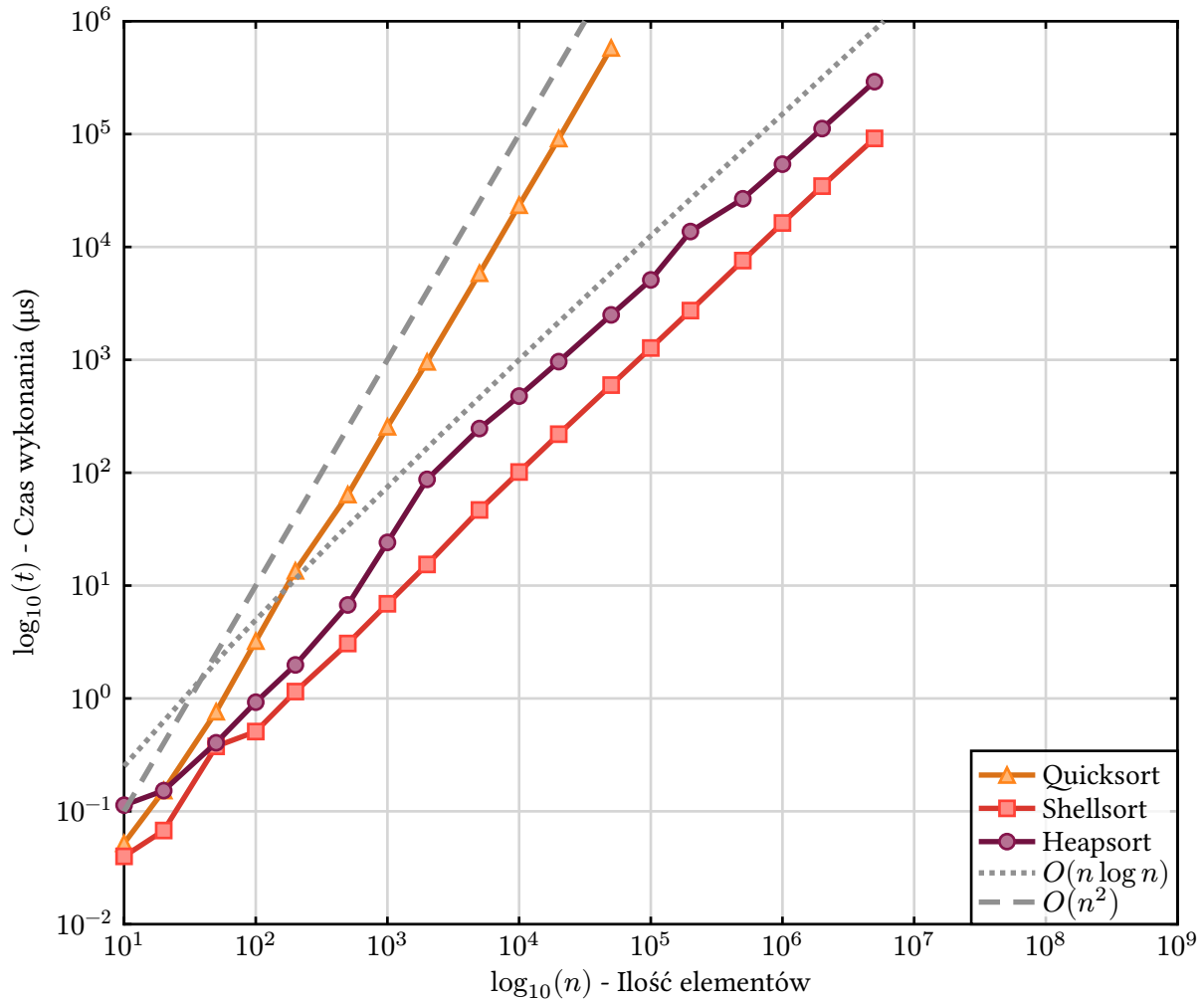
3.4.2. Dane posortowane malejąco (odwrócone)



Rysunek 15: Porównanie algorytmów efektywniejszych - dane posortowane malejąco (odwrócone) z liniami referencyjnymi $O(n \log n)$ i $O(n^2)$

Dla danych posortowanych malejąco (odwróconych) najlepiej wypada ShellSort, osiągając najkrótsze czasy wykonania. Na drugim miejscu znajduje się Heapsort. Oba te algorytmy bardzo dobrze radzą sobie z odwróconą tablicą, zachowując złożoność czasową $O(n \log n)$, co widać po ich krzywych biegnących wzdłuż linii referencyjnej. Zdecydowanie najwolniejszy w tym scenariuszu okazuje się QuickSort, dla którego takie ułożenie danych to najgorszy przypadek. Jego krzywa rośnie znacznie stromiej, układając się równolegle do przerywanej linii $O(n^2)$.

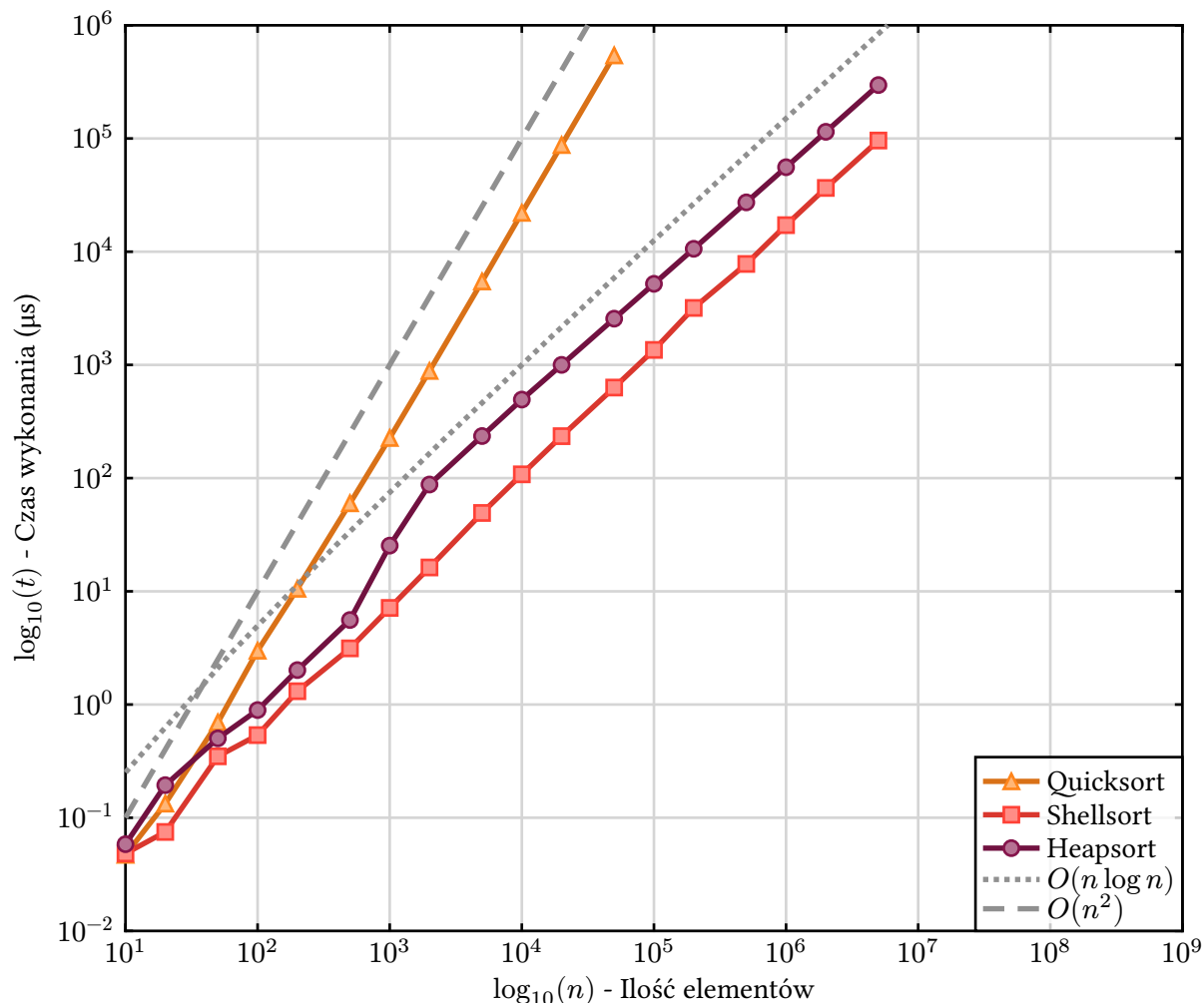
3.4.3. Dane posortowane rosnąco



Rysunek 16: Porównanie algorytmów efektywniejszych - dane posortowane rosnąco z liniami referencyjnymi $O(n \log n)$ i $O(n^2)$

W przypadku danych posortowanych rosnąco sytuacja jest bardzo podobna do tej z danymi odwróconymi. Ponownie najlepiej wypada ShellSort, osiągając najkrótsze czasy wykonania. Na drugim miejscu znajduje się Heapsort. Oba te algorytmy świetnie radzą sobie z już posortowaną tablicą, zachowując złożoność czasową $O(n \log n)$, co widać po ich krzywych biegnących wzdłuż linii referencyjnej. Zdecydowanie najwolniejszy w tym zestawieniu jest QuickSort. Podobnie jak przy danych malejących, ułożenie elementów w dobrej kolejności to dla niego najgorszy przypadek, przez co jego krzywa rośnie bardzo stromo, układając się równolegle do przerywanej linii $O(n^2)$.

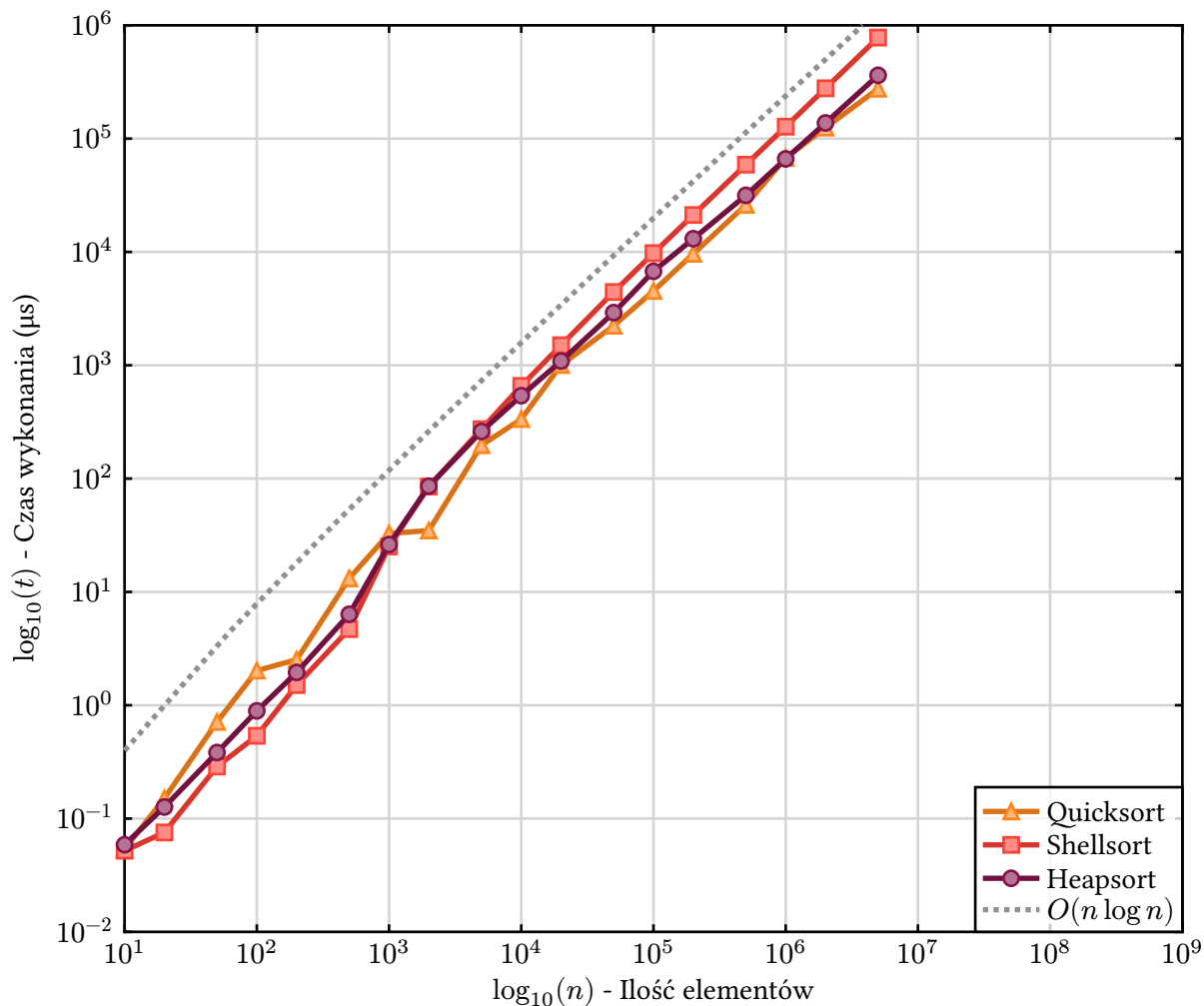
3.4.4. Dane prawie posortowane (sąsiednia wymiana)



Rysunek 17: Porównanie algorytmów efektywniejszych - dane prawie posortowane (sąsiednia wymiana) z liniami referencyjnymi $O(n \log n)$ i $O(n^2)$

Dla danych prawie posortowanych z sąsiednią wymianą sytuacja jest niemal identyczna jak w przypadku tablic ułożonych rosnąco lub malejąco. Ponownie najlepiej wypada Shellsort, osiągając najkrótsze czasy wykonania. Na drugim miejscu znajduje się Heapsort. Oba te algorytmy zachowują złożoność czasową $O(n \log n)$, co widać po ich krzywych biegnących wzdłuż linii referencyjnej. Quicksort z kolei ponownie radzi sobie zdecydowanie najgorzej. Niewielkie zaburzenia w postaci sąsiednich wymian to za mało, aby uchronić go przed najgorszym wariantem działania, przez co jego krzywa wciąż rośnie stromo wzdłuż przerywanej linii $O(n^2)$.

3.4.5. Dane prawie posortowane (globalna wymiana)

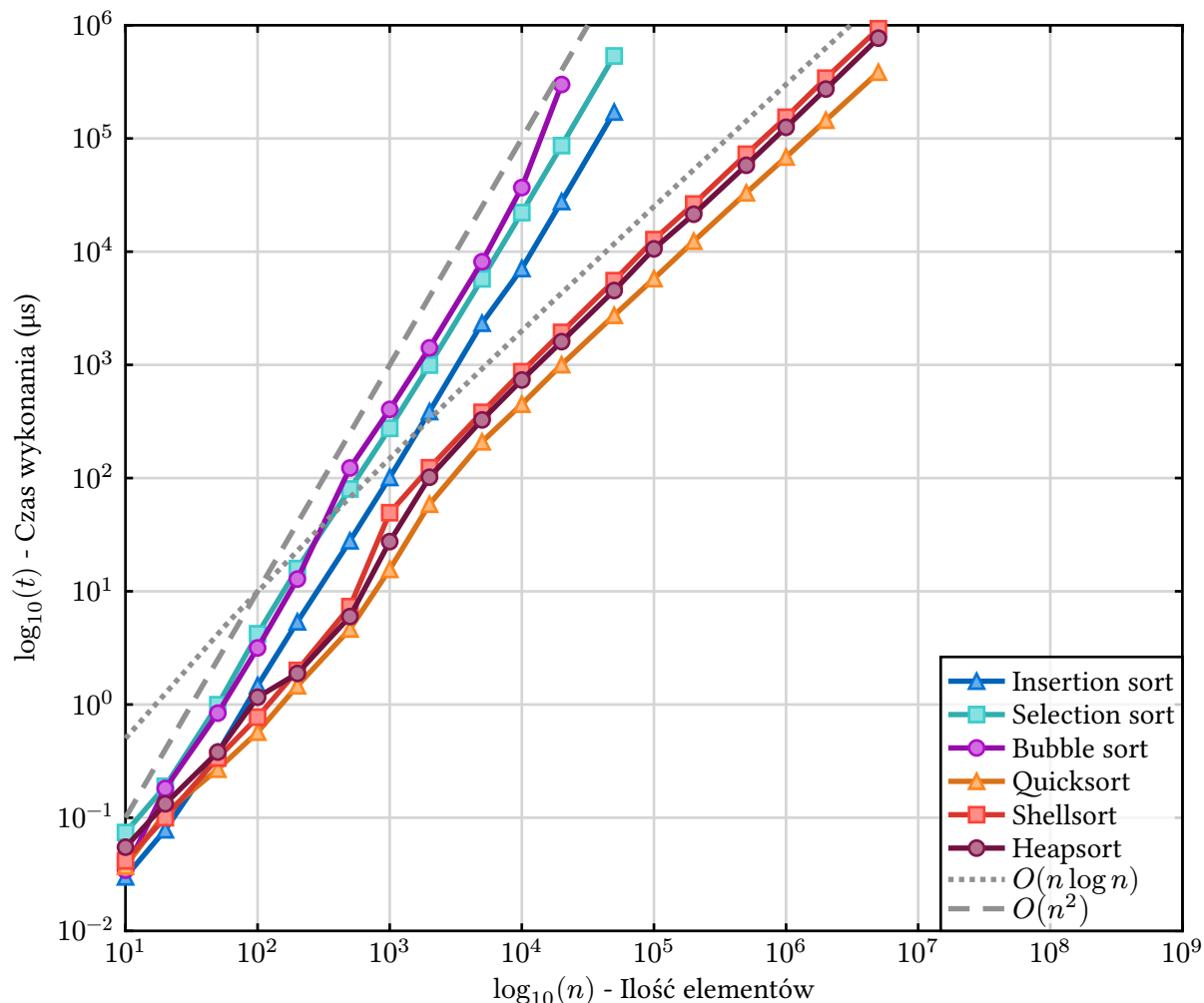


Rysunek 18: Porównanie algorytmów efektywniejszych - dane prawie posortowane (globalna wymiana) z linią referencyjną $O(n \log n)$

W scenariuszu z danymi prawie posortowanymi z globalną wymianą najlepiej wypada QuickSort, osiągając najkrótszy czas wykonania dla większych zestawów danych. Na drugim miejscu znajduje się Heapsort, natomiast najwolniejszy okazuje się ShellSort. Warto zauważyć, że globalne zaburzenia w danych skutecznie pozwalają algorytmowi QuickSort uniknąć najgorszego przypadku znanego z poprzednich wykresów. Dzięki temu krzywe wszystkich trzech metod rosną równoległe do linii referencyjnej, co potwierdza ich dobrą złożoność czasową na poziomie $O(n \log n)$ dla tego układu.

3.5. Porównanie algorytmów prostszych i efektywniejszych

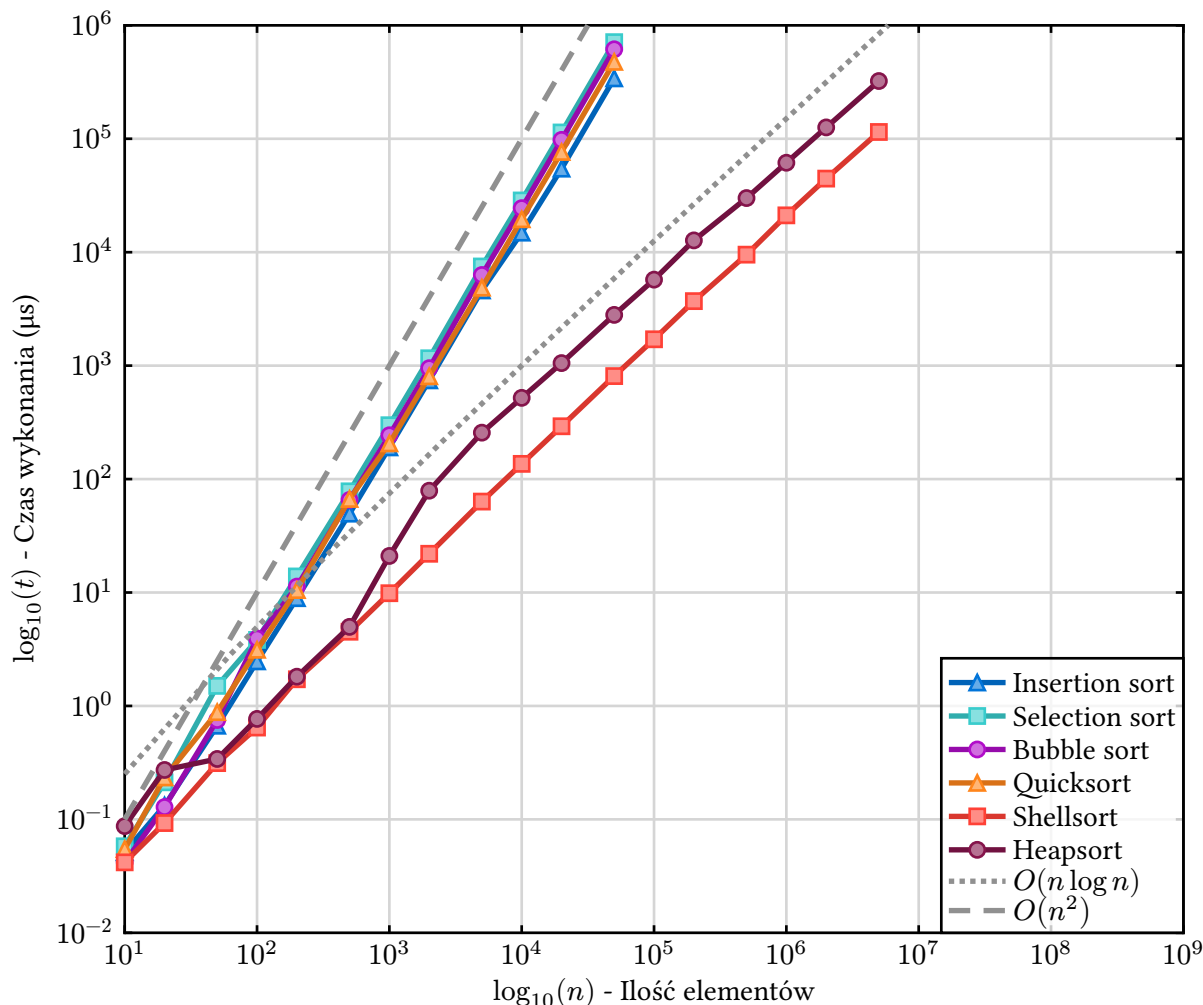
3.5.1. Dane losowe



Rysunek 19: Porównanie algorytmów prostych i efektywniejszych - dane losowe z liniami referencyjnymi $O(n \log n)$ i $O(n^2)$

W scenariuszu dla danych losowych widać wyraźny podział na dwie grupy algorytmów. Zdecydowanie najszybsze są metody efektywniejsze, wśród których najlepiej wypada Quicksort. Heapsort i Shellsort są nieznacznie wolniejsze, a krzywe całej tej trójki rosną wzdłuż linii referencyjnej $O(n \log n)$. Z kolei algorytmy proste radzą sobie znacznie gorzej. W tej grupie najszybszy jest Insertion sort, następnie Selection sort, natomiast najwolniejszy z całego zestawienia okazuje się Bubble sort. Krzywe algorytmów prostych rosną równoległe do linii referencyjnej $O(n^2)$, co idealnie pokazuje ogromną przepaść w wydajności między obiema grupami przy rosnącej liczbie elementów.

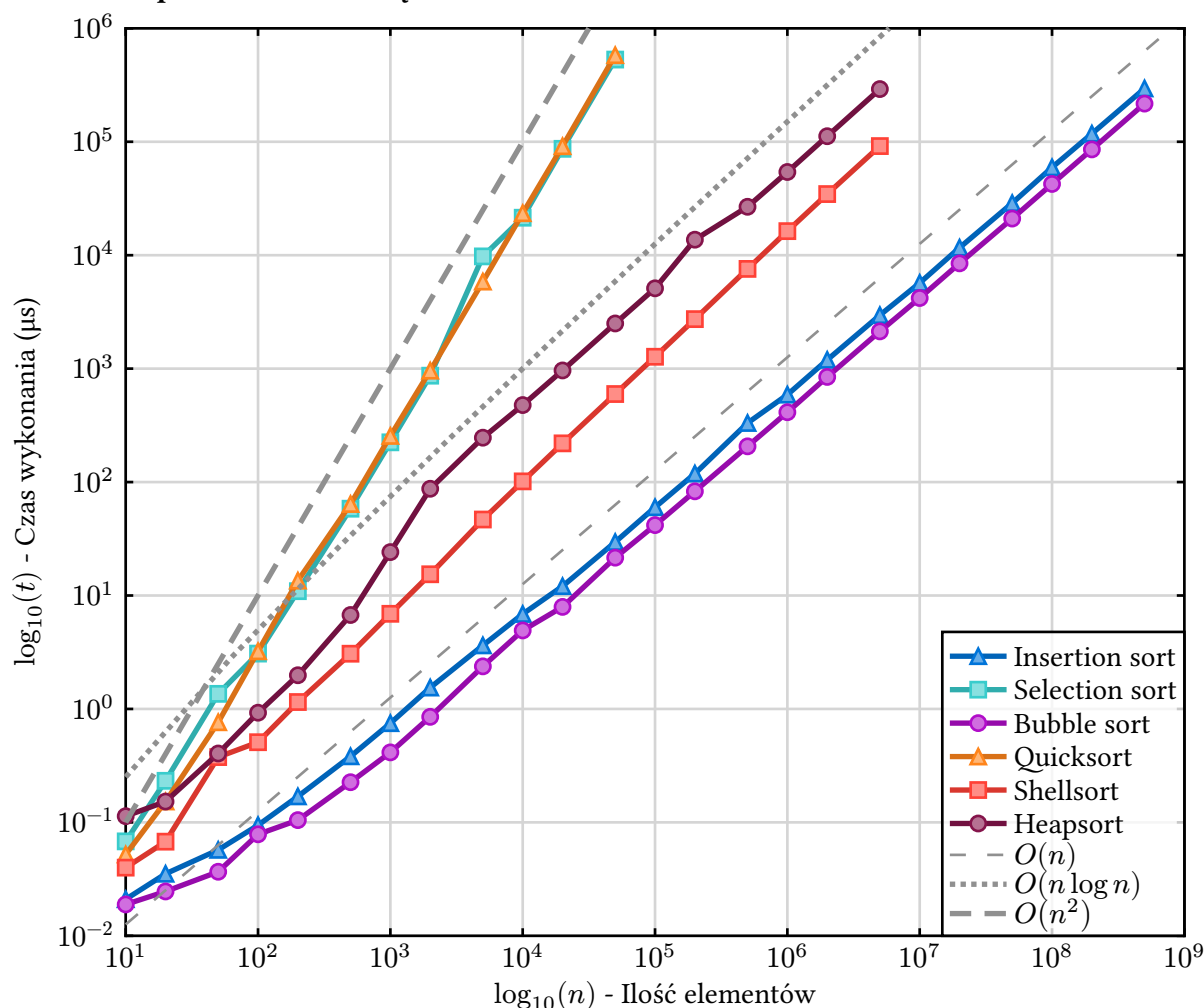
3.5.2. Dane posortowane malejąco (odwrócone)



Rysunek 20: Porównanie algorytmów prostych i efektywniejszych - dane posortowane malejąco (odwrócone) z liniami referencyjnymi $O(n \log n)$ i $O(n^2)$

Dla danych posortowanych malejąco (odwróconych) widać wyraźny podział na dwie grupy, ale z jednym bardzo ważnym wyjątkiem w stosunku do danych losowych. Zdecydowanie najszybsze pozostają Shellsort i Heapsort, z których to Shellsort radzi sobie najlepiej. Krzywe obu tych algorytmów rosną łagodnie wzdłuż linii referencyjnej $O(n \log n)$. Z kolei Quicksort dla tego ułożenia danych trafia na swój najgorszy przypadek i drastycznie zwalnia, spadając pod względem wydajności do grupy algorytmów prostych. W tej wolniejszej grupie najszybszy okazuje się Insertion sort, minimalnie wyprzedzając Quicksorta oraz Bubble sort. Najwolniejszy z całego zestawienia jest Selection sort. Krzywe wszystkich tych wolniejszych metod rosną stromo, równolegle do linii referencyjnej $O(n^2)$.

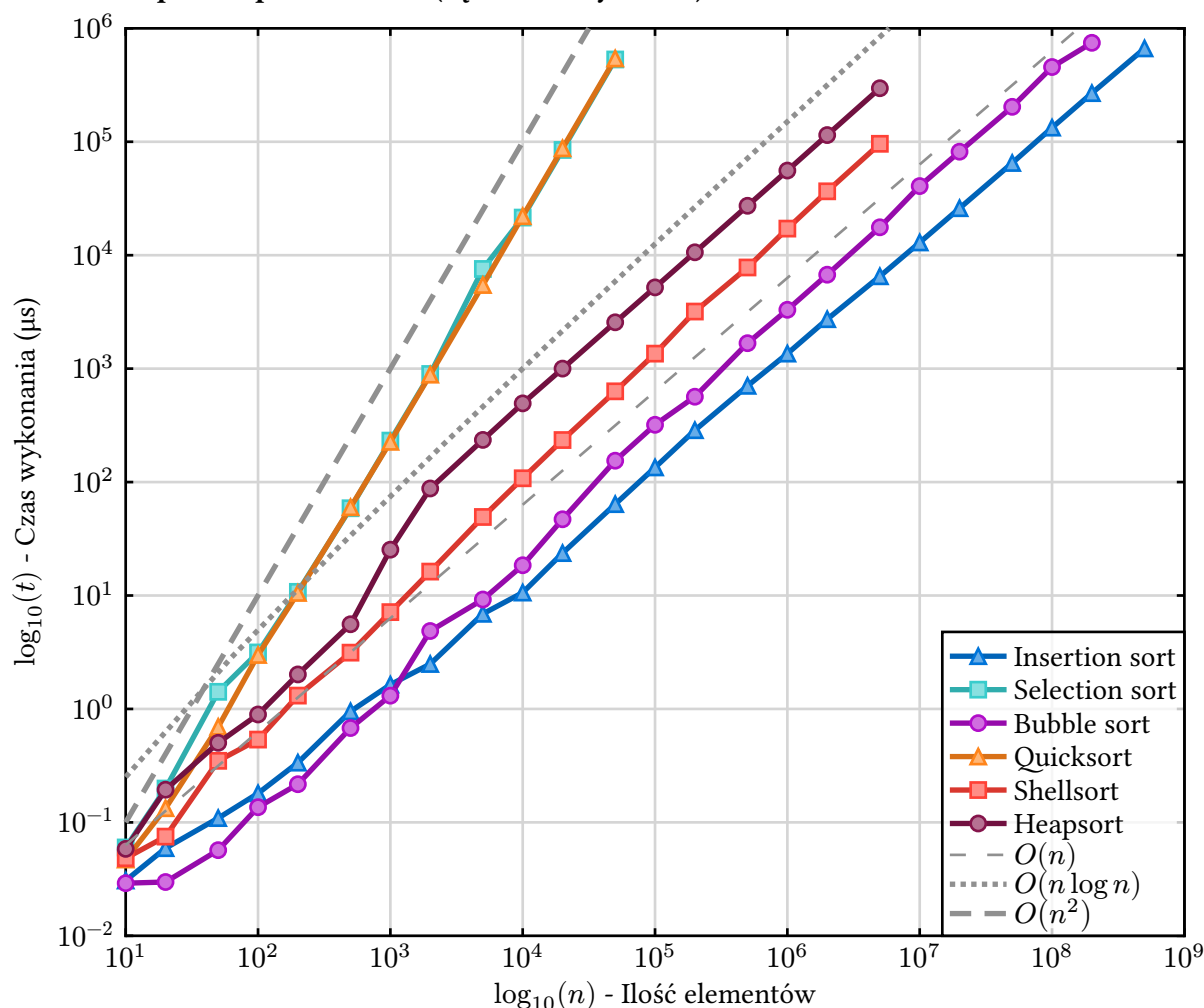
3.5.3. Dane posortowane rosnąco



Rysunek 21: Porównanie algorytmów prostych i efektywniejszych - dane posortowane rosnąco z liniami referencyjnymi $O(n)$, $O(n \log n)$ i $O(n^2)$

W przypadku danych posortowanych rosnąco najszybsze okazują się algorytmy z grupy prostych: Bubble sort oraz Insertion sort. Oba te algorytmy świetnie wykorzystują ułożenie danych i osiągają optymalny czas wzdłuż linii referencyjnej $O(n)$, przy czym Bubble sort jest minimalnie szybszy. Algorytmy z grupy efektywniejszych radzą sobie tutaj odczuwalnie gorzej. Shellsort i Heapsort zajmują środkową część wykresu, zachowując złożoność widoczną wzdłuż linii referencyjnej $O(n \log n)$, gdzie Shellsort jest wyraźnie szybszy od Heapsorta. Zdecydowanie najwolniejsze w tym zestawieniu są Quicksort oraz Selection sort, których krzywe niemal się pokrywają i rosną bardzo stromo wzdłuż linii referencyjnej $O(n^2)$. Dla Quicksorta jest to najgorszy możliwy przypadek, natomiast Selection sort jako jedyny algorytm prosty nie potrafi zaadaptować się do już posortowanego układu elementów i musi wykonać pełną liczbę porównań.

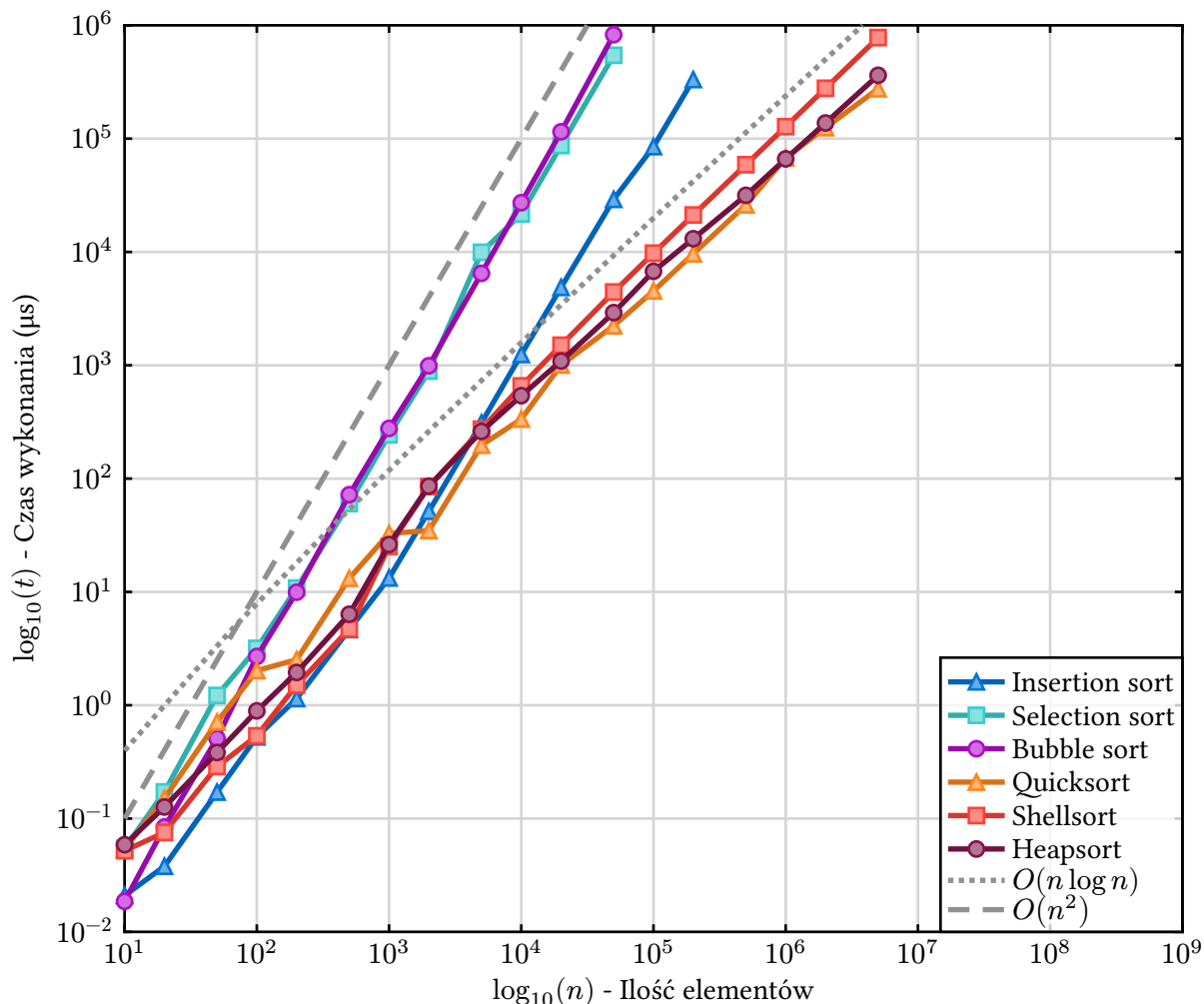
3.5.4. Dane prawie posortowane (sąsiednia wymiana)



Rysunek 22: Porównanie algorytmów prostych i efektywniejszych - dane prawie posortowane (sąsiednia wymiana) z liniami referencyjnymi $O(n)$, $O(n \log n)$ i $O(n^2)$

Dla danych prawie posortowanych z sąsiednią wymianą sytuacja wygląda niemal identycznie jak w przypadku danych posortowanych rosnąco. Zdecydowanie najszybsze pozostają algorytmy proste: Insertion sort oraz Bubble sort, które doskonale radzą sobie z niewielkimi lokalnymi zaburzeniami i osiągają czas działania wzdłuż linii referencyjnej $O(n)$. W tej dwójce Insertion sort jest minimalnie szybszy dla większych tablic. Algorytmy efektywniejsze – Shellsort i Heapsort – zajmują środkową pozycję na wykresie, rosnąc równoległe do linii referencyjnej $O(n \log n)$. W tej parze Shellsort ponownie wypada zauważalnie lepiej. Zdecydowanie najwolniejsze w całym zestawieniu są Quicksort oraz Selection sort, których krzywe pokrywają się i rosną stromo wzdłuż linii referencyjnej $O(n^2)$. Drobne zamiany sąsiednich elementów nie wystarczają, aby uchronić Quicksorta przed wydajnością bliską najgorszemu przypadkowi, a Selection sort wciąż musi wykonać pełną liczbę porównań niezależnie od niemal idealnego stanu początkowego tablicy.

3.5.5. Dane prawie posortowane (globalna wymiana)



Rysunek 23: Porównanie algorytmów prostych i efektywniejszych - dane prawie posortowane (globalna wymiana) z liniami referencyjnymi $O(n \log n)$ i $O(n^2)$

W scenariuszu dla danych prawie posortowanych z globalną wymianą ponownie widać wyraźny podział na dwie grupy algorytmów. Zdecydowanie najszybsze są metody efektywniejsze, wśród których najlepiej radzi sobie Quicksort. Zaraz za nim znajduje się Heapsort, a najwolniejszy z tej trójki okazuje się Shellsort. Krzywe tych trzech algorytmów rosną wzdłuż linii referencyjnej $O(n \log n)$. Globalne zaburzenia w ułożeniu elementów sprawiają, że Quicksort skutecznie unika swojego najgorszego przypadku i osiąga wysoką wydajność. Algorytmy proste radzą sobie z tymi danymi znacznie gorzej, a ich krzywe rosną stromo, układając się równolegle do linii referencyjnej $O(n^2)$. W tej wolniejszej grupie najszybszy jest Insertion sort. Zdecydowanie najwolniejszy w całym zestawieniu są natomiast Selection sort oraz Bubble sort, których krzywe niemal całkowicie się pokrywają.

4. Algorytmy niekonwencjonalne

4.1. Stooge sort

Zasada działania algorytmu

Stooge sort to rekurencyjny algorytm sortowania, który działa na zasadzie trzykrotnego sortowania nakładających się podtablic. Dla danego zakresu tablicy algorytm najpierw sprawdza, czy pierwszy i ostatni element są w złej kolejności - jeśli tak, zamienia je miejscami. Następnie, jeśli zakres ma co najmniej 3 elementy, rekurencyjnie sortuje

- pierwsze 2/3 tablicy
- ostatnie 2/3 tablicy
- ponownie pierwsze 2/3 tablicy

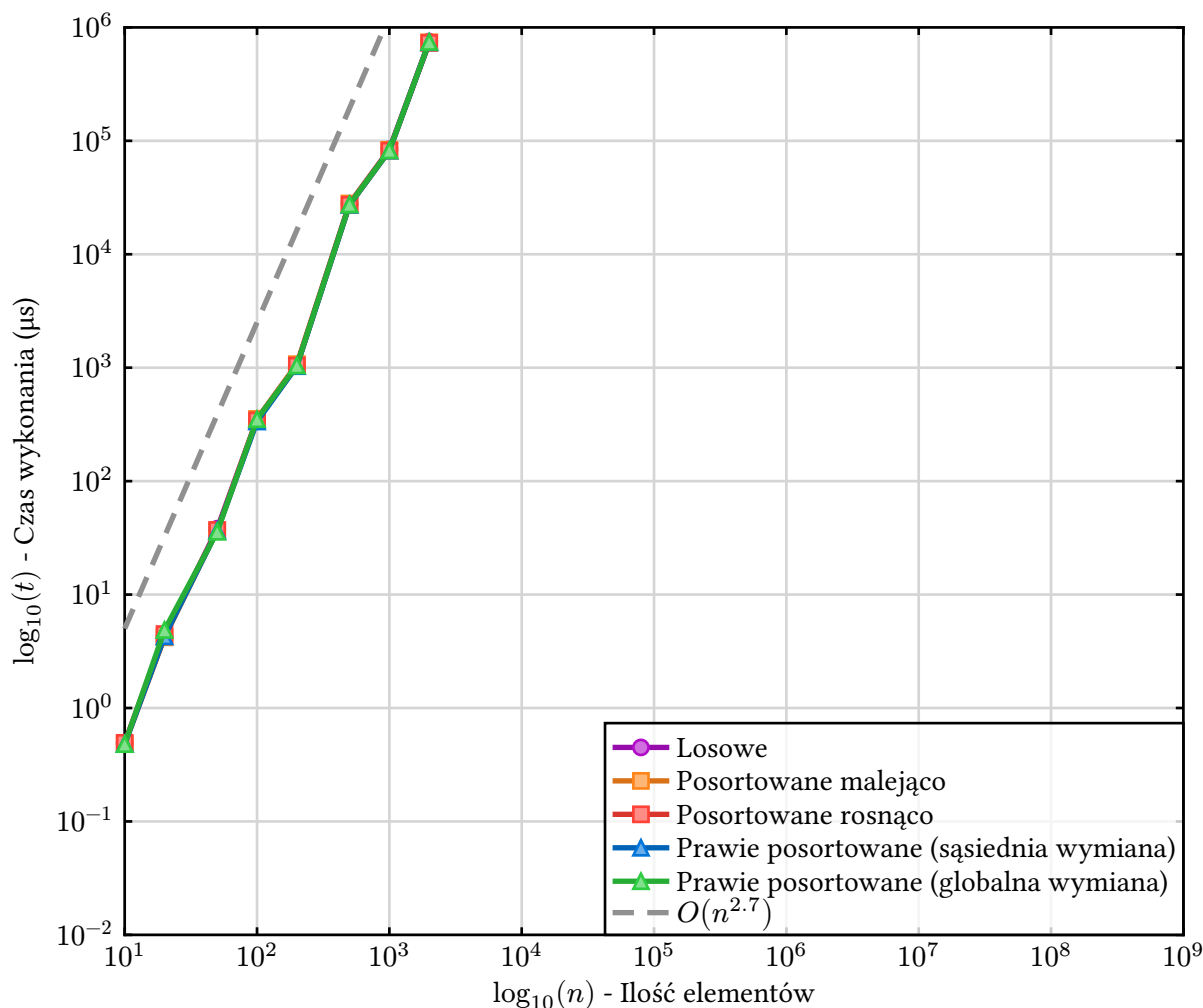
Podwójne sortowanie pierwszej części jest konieczne, by elementy przesunięte przez sortowanie środkowego fragmentu znalazły swoje ostateczne miejsca.

Algorytm 7: Stooge sort

```
1: procedure STOOGESORT(arr, l, r)
2:   if arr[l] > arr[r] then
3:     | zamień arr[l] z arr[r]
4:   end
5:
6:   if r - l + 1 > 2 then
7:     |  $t \leftarrow \lfloor \frac{r-l+1}{3} \rfloor$ 
8:     STOOGESORT(arr, l, r - t)
9:     STOOGESORT(arr, l + t, r)
10:    STOOGESORT(arr, l, r - t)
11:   end
12: end
```

Hipotezy badawcze dla poszczególnych scenariuszy

1. **Dane losowe:** Oczekuję czasu między $O(n^2)$ a $O(n^3)$. Algorytm jest niezdolny do skrócenia rekursji niezależnie od wejścia.
2. **Dane posortowane malejąco (odwrócone):** Algorytm nie potrafi wykryć, że dane są odwrócone, i wykona pełną liczbę wywołań rekurencyjnych. Spodziewam się czasu zbliżonego do danych losowych.
3. **Dane posortowane rosnąco:** Mimo że dane są już posortowane, Stooge sort nadal wykonuje wszystkie trzy wywołania rekurencyjne na każdym poziomie. Jedyne oszczędności to brak zamian pierwszego i ostatniego elementu. Czas powinien być minimalnie krótszy niż dla reszty.
4. **Dane prawie posortowane (sąsiednia wymiana):** Oczekuję wyników praktycznie identycznych z danymi losowymi.
5. **Dane prawie posortowane (globalna wymiana):** Brak możliwości adaptacji do struktury danych oznacza, że wynik będzie znów zbliżony do pozostałych scenariuszy.



Rysunek 24: Czas wykonania Stoooge sort - porównanie scenariuszy danych wejściowych z linią referencyjną $O(n^{2.7})$

Rysunek 24 potwierdza większość z postawionych hipotez. Wszystkie pięć krzywych układa się identycznie i rośnie równolegle do linii referencyjnej $O(n^{2.7})$, co jest bezpośrednim dowodem na nieadaptywność algorytmu. Warto zwrócić uwagę na bardzo ograniczony zakres przetestowanych rozmiarów tablic - ze względu na ogromną stałą ukrytą w złożoności, Stoooge sort staje się praktycznie bezużyteczny już przy kilkuset elementach. Dla porównania, przy tych samych rozmiarach Quicksort czy Heapsort działają tysiące razy szybciej.

Czasowa złożoność obliczeniowa

- **Najgorszy, średni i najlepszy przypadek:** $O(n^{\log(3)/\log(\frac{3}{2})}) = O(n^{2.709...})$ - Złożoność wynika bezpośrednio z postaci rekurencji: $T(n) = 3T(\frac{2}{3}n) + O(1)$. Algorytm jest całkowicie nieadaptywny i zawsze wykonuje tę samą liczbę wywołań rekurencyjnych niezależnie od wejścia.

4.2. Thanos sort

Zasada działania algorytmu

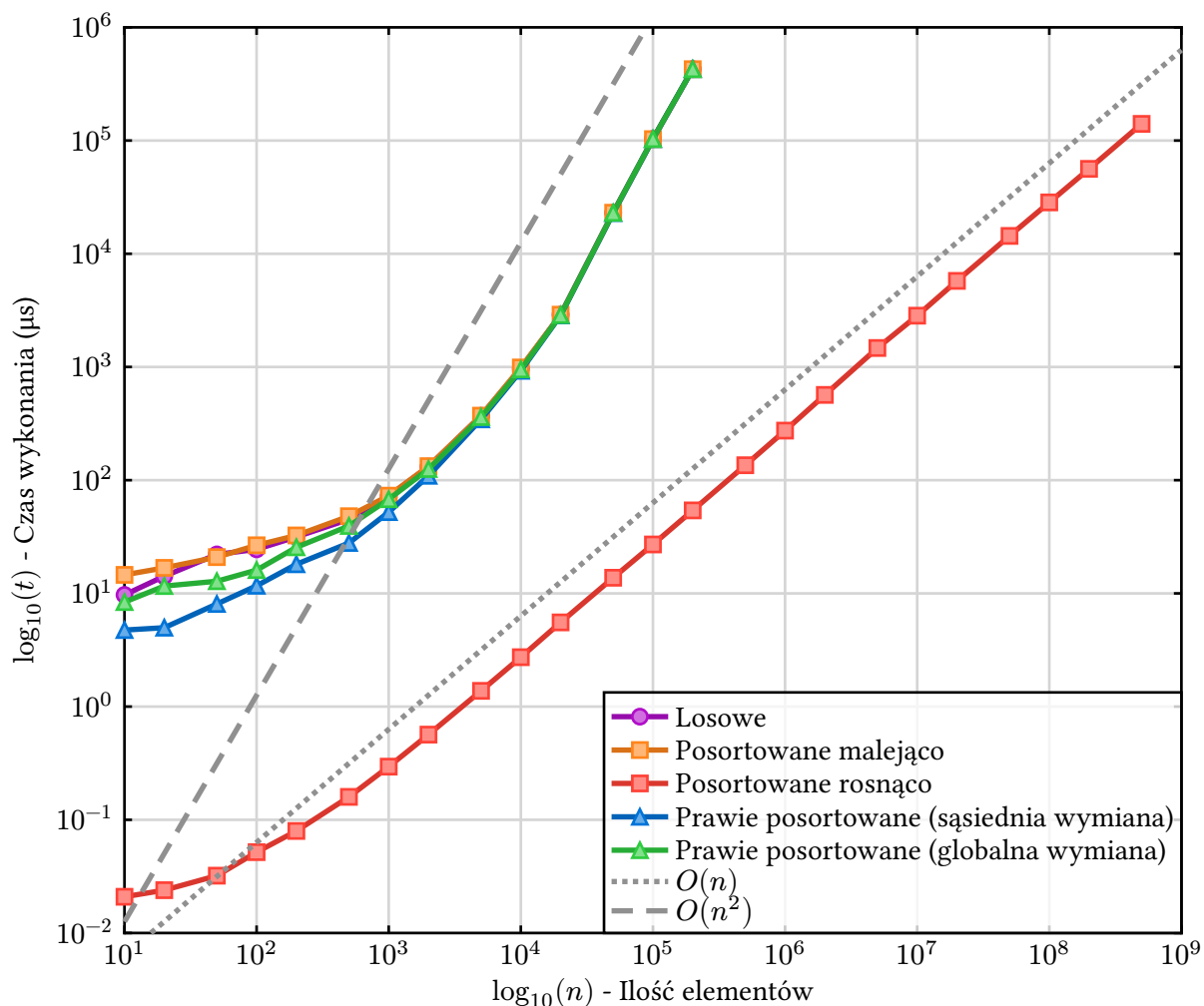
Thanos sort to żartobliwy algorytm inspirowany postacią z filmów Marvela. Polega na sprawdzaniu, czy tablica jest już posortowana - jeśli tak, zakończ. Jeśli nie, usuń połowę elementów i sprawdź ponownie. Proces powtarza się, aż pozostałe elementy będą ułożone w porządku rosnącym. Algorytm zawsze zakończy działanie, jednak kosztem niekompletności danych - wynikowa tablica jest jedynie posortowanym podzbiorem oryginału, zazwyczaj o wiele mniejszym niż dane wejściowe.

Algorytm 8: Thanos sort

```
1: function IS-SORTED(arr, n)
2:    $i \leftarrow 0$ 
3:   while  $i < n - 1$  do
4:     if  $\text{arr}[i] > \text{arr}[i + 1]$  then
5:       return false
6:     end
7:      $i \leftarrow i + 1$ 
8:   end
9:   return true
10: end
11:
12: procedure THANOS SORT(arr, n)
13:   while not is-sorted[arr, n] do
14:     usuń połowę losowych elementów z arr
15:      $n \leftarrow \lfloor \frac{n}{2} \rfloor$ 
16:   end
17: end
```

Hipotezy badawcze dla poszczególnych scenariuszy

1. **Dane losowe:** Oczekiwany czas to $O(n \log n)$, ponieważ każda runda usuwa połowę elementów.
2. **Dane posortowane malejąco (odwrócone):** Po odrzuceniu połowy tablicy układ staje się niemal losowy, spodziewany czas zbliżony do danych losowych.
3. **Dane posortowane rosnąco:** Algorytm od razu sprawdzi, że dane są posortowane. Spodziewany najszybszy czas $O(n)$.
4. **Dane prawie posortowane (sąsiednia wymiana):** Łatwo uzyskać posortowany zbiór po wyrzuceniu kilku elementów. Czas lepszy niż dla losowych, gorszy niż dla rosnących.
5. **Dane prawie posortowane (globalna wymiana):** Odległe zamiany mocniej psują tablicę. Czas zbliżony do danych losowych.



Rysunek 25: Czas wykonania Thanos sort - porównanie scenariuszy danych wejściowych z liniami referencyjnymi $O(n)$ i $O(n^2)$

Rysunek 25 w większości obala postawione hipotezy, jedyna która była trafna to dla danych już posortowanych. Reszta prawdopodobnie nie sprawdziła się przez to jak dużym wysiłkiem okazuje się być usunięcie losowej połowy danych. Możemy też zauważyć, że wszystkie scenariusze danych poza danymi posortowanymi rosnąco tworzą łuk, który stopniowo wygładza się do $O(n^2)$, jest to najprawdopodobniej spowodowane dodatkowym overheadem pobierania losowych liczb i usuwania na ich podstawie elementów tablicy.

Czasowa złożoność obliczeniowa

- **Najgorszy i średni przypadek:** $O(n^2)$ - Chociaż algorytm zmniejsza pulę elementów o połowę w każdym kroku, ukryty koszt ciągłego modyfikowania wektora i przesuwania w nim pamięci ostatecznie sprowadza jego wydajność do kwadratowej.
- **Najlepszy przypadek:** $O(n)$ - Algorytm wykonuje zaledwie jedno przejście weryfikacyjne przez idealnie posortowaną tablicę, funkcja `is-sorted` zwraca `true`, a algorytm kończy działanie bez usunięcia ani jednego elementu.

4.3. Stalin sort

Zasada działania algorytmu

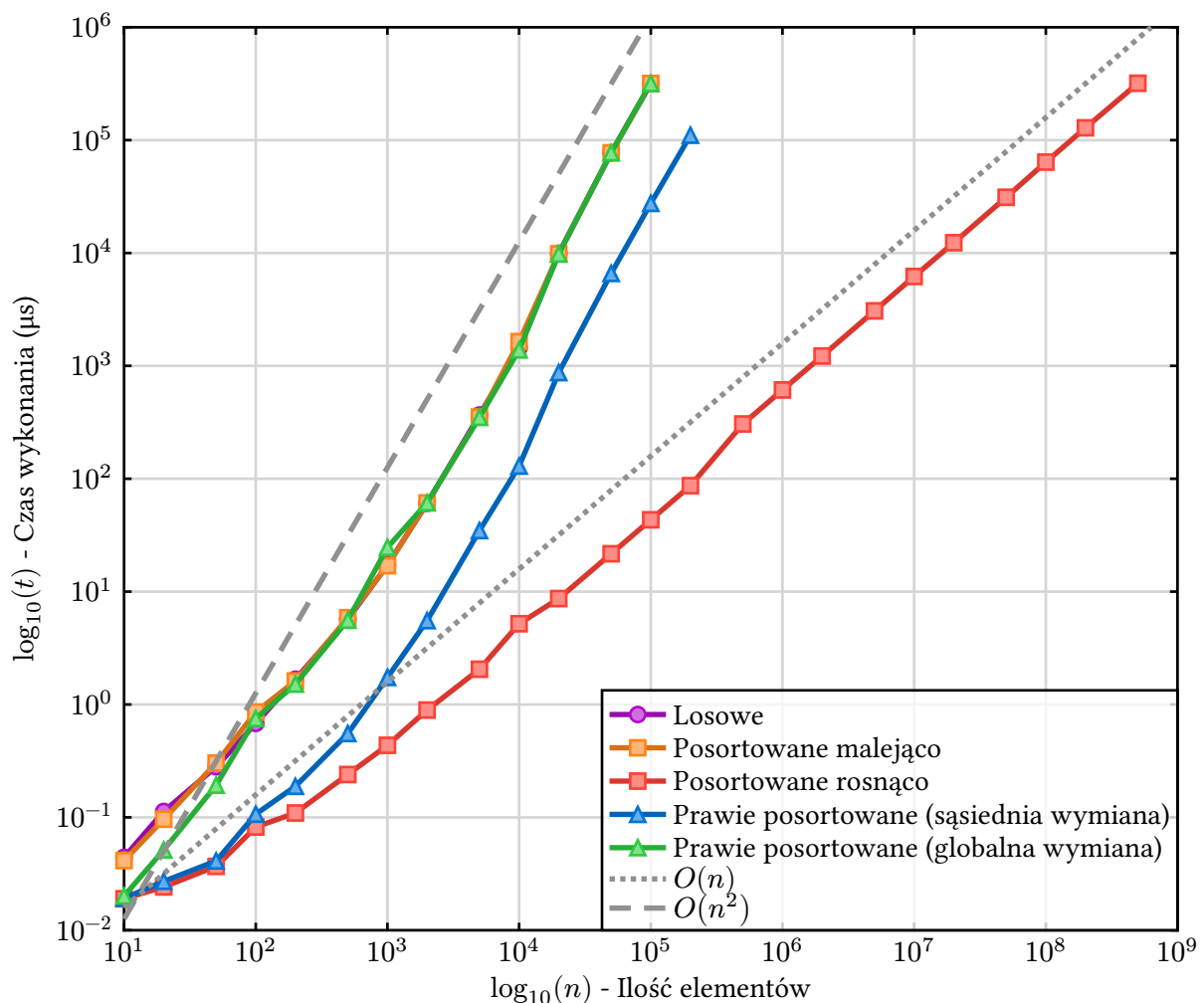
Stalin sort to kolejny żartobliwy algorytm, którego „sortowanie” polega na usuwaniu elementów psujących porządek zamiast ich przestawiania - nawiązując tym samym do stalinowskiej metody rozwiązywania problemów. Algorytm wykonuje jedno liniowe przejście przez tablicę, śledząc dotychczasowe maksimum. Jeśli bieżący element jest większy lub równy maksimum - zostaje zachowany i maksimum jest aktualizowane. Jeśli jest mniejszy - zostaje bezpowrotnie usunięty. W wyniku tego procesu otrzymujemy podciąg oryginalnej tablicy posortowany niemalejąco, jednak zazwyczaj krótszy niż oryginał.

Algorytm 9: Stalin sort

```
1: procedure STALIN SORT(arr, n)
2:   max ← arr[0]
3:   i ← 1
4:   while i < n do
5:     if arr[i] ≥ max then
6:       max ← arr[i]
7:     else
8:       usuń arr[i] z tablicy
9:       i ← i - 1
10:    end
11:    i ← i + 1
12:  end
13: end
```

Hipotezy badawcze dla poszczególnych scenariuszy

1. **Dane losowe:** Oczekiwany czas $O(n)$, algorytm jednorazowo przejdzie przez tablicę i zachowa tylko rosnące liczby.
2. **Dane posortowane malejąco (odwrócone):** Zachowa tylko pierwszą liczbę, a resztę usunie. Spodziewany czas $O(n)$.
3. **Dane posortowane rosnąco:** Nic nie zostanie usunięte. Spodziewany czas $O(n)$.
4. **Dane prawie posortowane (sąsiednia wymiana):** Usunie bardzo mało elementów, czas $O(n)$.
5. **Dane prawie posortowane (globalna wymiana):** Usunie więcej elementów, ale spodziewany czas to nadal $O(n)$.



Rysunek 26: Czas wykonania Stalin sort - porównanie scenariuszy danych wejściowych z liniami referencyjnymi $O(n)$ i $O(n^2)$

Początkowe hipotezy zakładały, że algorytm we wszystkich przypadkach osiągnie optymalny czas $O(n)$. Wykres pokazuje jednak, że dzieje się tak tylko dla danych rosnących, ponieważ nic z nich nie usuwamy. W pozostałych scenariuszach czas drastycznie rośnie, docierając do poziomu $O(n^2)$. Przyczyna jest identyczna jak w Thanos sort. Przykładowo, dla danych malejących algorytm usuwa prawie wszystkie elementy. Każde wyrzucenie wartości wymusza na programie przesuwanie kolejnych liczb w lewo. To ciągłe przemieszczanie bloków pamięci zabija całą szybkość algorytmu.

Czasowa złożoność obliczeniowa

- **Najgorszy i średni przypadek:** $O(n^2)$ - Wynika to z technicznego kosztu usuwania elementów z tablicy w miejscu. Gdyby modyfikacja opierała się na budowaniu zupełnie nowej tablicy wynikowej lub korzystała z list powiązanych, algorytm z powodzeniem działałby w oczekiwanym czasie $O(n)$.
- **Najlepszy przypadek:** $O(n)$ - Brak konieczności usuwania jakiegokolwiek elementu, dane już posortowane.